Kanboard BA Dökümanı - QA Test Analizi (Full Scope)

Business Analysis & Test Scope Specification

Ürün / Product	Kanboard (Open Source)
Doküman / Document	BA & QA Test Analizi
Sürüm / Version	1.0
Tarih / Date	2026-03-05
Hazırlayan / Prepared by	Hasan Emre
Gözden Geçiren / Reviewed by	QA Lead / Tech Lead

Revizyon Geçmişi (Revision History)

Sürüm	Tarih	Değişiklik Özeti	Yapan
1.0	2026-03-05	İlk yayın (initial release)	Hasan Emre

1. Purpose

Bu Business Analysis (BA) dokümanının amacı, Kanboard uygulamasının işlevsel (functional) ve işlevsel olmayan (non-functional) gereksinimlerini sistematik olarak analiz etmek ve test süreçlerine referans oluşturacak kapsamlı bir çerçeve sağlamaktır.

Bu doküman kapsamında:

- Sistem modülleri ve işlevleri tanımlanmıştır,
- Actor ve role-based access control (RBAC) yapısı analiz edilmiştir,
- Business rules belirlenmiştir,
- Yetkilendirme ve erişim kontrolleri değerlendirilmiştir,
- API katmanı incelenmiştir,
- Riskli alanlar ve test öncelikleri ortaya konmuştur.

Bu belge, QA Lead ve teknik ekip için referans niteliğinde olup; test planlama, test case oluşturma, authorization matrix hazırlanması ve API test kapsamının belirlenmesi süreçlerinde temel doküman olarak kullanılacaktır.

1.1 Baseline Sürüm ve Referanslar

Bu BA dokümanı, Kanboard resmi dokümantasyonu (Kanboard v1.x) esas alınarak hazırlanmıştır.

Doküman hazırlanma tarihi: 05.03.2026.

Bu tarihte yayınlanan en güncel stabil sürüm: Kanboard 1.2.50 (07.02.2026).

Hedef sürüm: Kurulu Kanboard sürümü (1.2.x) dokümanda ayrıca kayıt altına alınmalıdır.

Referanslar: https://docs.kanboard.org/v1/ , https://kanboard.org/releases.html

2. System Overview

Kanboard, açık kaynaklı (open-source), web tabanlı bir Kanban proje yönetim uygulamasıdır. Sistem, görsel board yapısı üzerinden proje ve görevlerin (task) yönetilmesini sağlar ve iş akışlarının (workflow) durum bazlı takibini destekler.

Uygulamanın temel mimari ve işleyiş özellikleri aşağıdaki gibidir:

- Web-based mimari: Kullanıcılar tarayıcı üzerinden sisteme erişir.
- Role-based access control (RBAC): Kullanıcılar tanımlı roller doğrultusunda yetkilendirilir.
- Project-centric yapı: Tüm task'lar bir project altında oluşturulur ve yönetilir.
- Visual board management: Task'lar column yapısı üzerinde drag & drop ile taşınır.
- Subtask ve time tracking desteği: Task'lar alt görevlerle detaylandırılabilir.
- File attachment ve metadata desteği: Task ve project seviyesinde dosya eklenebilir.
- JSON-RPC API desteği: Sistem, dış sistem entegrasyonları için API katmanı sağlar.
- Plugin ve extensibility yapısı: Eklenti mekanizması ile genişletilebilir mimari sunar.

Dokümantasyon yapısı üç ana kategori altında organize edilmiştir:

- Using Kanboard – Kullanıcı işlemleri ve uygulama içi fonksiyonlar
- Administration – Sistem kurulumu, yapılandırma ve güvenlik ayarları
- API Reference – JSON-RPC tabanlı API prosedürleri ve endpoint yapısı

Bu analiz kapsamında sistem, kullanıcı işlemleri, yetkilendirme yapısı ve API katmanı perspektifinden ele alınacaktır.

3. Scope

3.1 In Scope

Bu analiz kapsamında Kanboard uygulamasının tüm temel modülleri ve destekleyici bileşenleri incelenmiştir.

Aşağıdaki alanlar kapsam dahilindedir:

A) Core Application Modules

- Authentication & Session Management
- Project Management
- Task Management
- Subtasks
- Columns & Swimlanes
- Categories & Tags
- File Attachments
- Time Tracking
- Search & Filters
- Notifications (Email, RSS, iCalendar)
- Automatic Actions

B) User & Access Management

1. User Management
2. Groups
3. Role-based access control (RBAC)
4. Project Permissions
5. Authorization matrix doğrulaması

C) Administration

1. Configuration Settings
2. Security Settings
3. Background Workers
4. Email Configuration

5. Database configuration (conceptual level)
6. Performance considerations
7. Upgrade & Maintenance processes

D) API Layer (JSON-RPC)

1. Authentication procedures
2. Project API procedures
3. Task API procedures
4. User API procedures
5. Subtask API procedures
6. File API procedures
7. Permission API procedures
8. Error handling (400, 401, 404 scenarios)

E) Non-Functional Considerations

1. Security validation
2. Authorization bypass kontrolü
3. Input validation
4. Data integrity
5. Session timeout
6. File upload güvenliği

3.2 Out of Scope

Bu analiz kapsamında aşağıdaki alanlar hariç tutulmuştur:

1. Production environment load testing
2. Real infrastructure stress testing
3. Third-party external integrations
4. Custom plugin geliştirme

(Not: Mevcut plugin fonksiyonları functional olarak değerlendirilmiştir, ancak plugin development lifecycle analiz edilmemiştir.)

4. Actors & Role Structure (Revize Final)

4.1 Role Model (Yetkilendirme Seviyeleri)

Kanboard'da yetkilendirme iki seviyede ele alınır:

1. **Uygulama (Application) seviyesinde roller**: Sistemin genelinde geçerlidir.
2. **Proje (Project) seviyesinde roller**: Sadece ilgili proje içinde geçerlidir.

4.2 Uygulama Rolleri (Application Roles)

Her kullanıcı aşağıdaki uygulama rollerinden birine sahiptir:

Role	Açıklama
Yönetici (Administrator)	Her şeye erişim
Müdür (Manager)	Ekip projeleri oluşturabilir, ancak uygulama ayarlarını değiştiremez
Kullanıcı (User)	Sadece özel proje oluşturabilir

BA Notu: "Müdür (Manager)" ile "Proje Müdürü (Project Manager)" farklı kavramlardır; biri uygulama rolü, diğeri proje rolüdür.

4.3 Proje Rolleri (Project Roles)

Her ekip projesinde, her kullanıcıya ve gruba farklı rol atanabilir:

Role	Açıklama
Proje Müdürü (Project Manager)	Proje ayarlarını değiştirebilir; Gantt grafiğine ve raporlara erişebilir
Proje Üyesi (Project Member)	Task (görev) oluşturabilir ve board'u kullanabilir
Proje Görüntüleyici (Project Viewer)	Board ve task'lara salt-okunur (read-only) erişim sağlar

4.4 Access Model (Proje Erişimi ve İzinler)

1. Her proje diğer projelerden izoledir; proje erişimi **proje sahibi** tarafından yetkilendirilir.
2. Her kullanıcıya ve her gruba projede ayrı rol atanabilir.
3. Rol atamaları: **Proje Ayarları > İzinler** ekranında yönetilir.
4. **Özel (private/personal) projeler için izin tanımlanamaz**; erişim modelinde kullanıcı yönetimi yoktur.

4.5 Proje Tipleri ve Rol Etkisi

Dokümantasyona göre iki proje tipi vardır: **Takım Projesi** ve **Özel Proje**.

1. **Takım Projesi**: kullanıcı ve grup yönetimi vardır.

2. **Özel Proje**: kullanıcı yönetimi yoktur; projeye yalnızca sahibi ve yöneticiler erişebilir.

Not (Doküman tutarlılığı):

1. Bir sayfa "Takım projesini yalnızca yöneticiler ve proje yöneticileri oluşturabilir" der.

2. Başka sayfa "Takım projesini yalnızca yöneticiler ve uygulama yöneticileri (Manager) oluşturabilir" der.
   BA açısından bunu "doğrulanacak business rule" olarak ele alacağız ve uygulamada yetki davranışını testle netleyeceğiz.

4.6 User Types (Kullanıcı Tipleri)

Kanboard'da iki kullanıcı tipi bulunur:

1. **Yerel (Local) kullanıcı**: Şifre Kanboard veritabanında tutulur.

2. **Uzak (Remote) kullanıcı**: Kimlik bilgileri başka bir sistem tarafından yönetilir (örn. LDAP, reverse proxy).

4.7 User & Group Management (Yönetim)

1. Yeni kullanıcı eklemek için **yönetici** olmak gerekir.

2. Grup oluşturma ve kullanıcıları gruba atama işlemleri **yalnızca yöneticiler** tarafından yapılır.

3. Proje izinleri sayfasından gruplara erişim yetkisi verilebilir.

4.8 Custom Project Roles (Özel Proje Rolleri)

Özel proje rolleri, **Proje Üyesi (Project Member)** rolünden türeyen ve kullanıcıya kısıtlar uygulayan proje bazlı rollerdir (örn. belirli sütunlar arası taşıma kısıtı).

5. Functional Analysis ve Test Kapsamı

5.1 Project Management Module (Proje Yönetimi)

5.1.1 Overview

Kanboard'da çalışma modeli "project-centric"tir: İşlerin yönetimi projeler (project) üzerinden yürütülür. Sistem birden fazla projeyi destekler ve projeler üzerinde task'lar farklı görünümler (views) ile yönetilir.

5.1.2 Project Types (Proje Tipleri)

Sistemde iki temel proje tipi bulunur:

1. **Team Project (Takım Projesi):** Kullanıcı ve grup yönetimi bulunan proje tipidir. Dokümantasyona göre yalnızca **yöneticiler (Administrator)** ve **uygulama yöneticileri (Manager)** team project oluşturabilir.

2. **Private Project (Özel Proje):** Tek bir kişiye ait proje tipidir; **kullanıcı yönetimi yoktur** ve dokümantasyona göre herkes private project oluşturabilir.

BA Validation Item (Doküman tutarlılığı / doğrulanacak kural):
"Team project'i kim oluşturabilir?" ifadesi dokümanda farklı şekilde de geçmektedir: "Yalnızca yöneticiler ve proje yöneticileri bu projeleri oluşturabilir."
→ Bu nedenle uygulamadaki gerçek davranış test ile doğrulanmalı ve yetki matrisi buna göre finalize edilmelidir.

5.1.3 Core Functionalities (Temel Fonksiyonlar)

A) Create Project (Proje Oluşturma)

1. Sistem birden fazla proje yönetimini destekler.

2. Proje oluşturma sırasında proje tipi seçimi (team/private) mevcuttur.

3. Yeni proje oluştururken, başka bir projeden bazı özellikler **çoğaltılabilir (duplicate/copy)**: **permissions, actions, swimlanes, categories, tasks**.

B) Edit Project (Projeyi Düzenleme)

1. Projeler yeniden adlandırılabilir ve devre dışı bırakılabilir.

2. **Start date / end date** bilgisi proje **Gantt** grafiği için kullanılır.

3. **Description** alanı, board ve proje listeleme sayfasında tooltip olarak görünebilir.

4. **Administrator** ve **Project Manager**, "Private Project" işaretlemesini değiştirerek private ↔ team dönüşümü yapabilir.

5. Proje “private” yapıldığında mevcut kullanıcıların erişimi otomatik olarak kaldırılmaz; kullanıcı listesinin ihtiyaca göre düzenlenmesi gerekir.

C) Project Views (Proje Görünümleri)

1. Proje görevleri şu görünümlerle görselleştirilebilir: **Board, Calendar, List, Gantt**.
2. Her görünüm, üstteki **filter/search** kutusunun sonucunu gösterir; arama motoru gelişmiş sözdizimini (advanced syntax) destekler.
3. Board görünümünde task’lar sütunlar arasında **drag & drop** ile taşınabilir.

D) Project Settings / Defaults (Proje Ayarları ve Varsayılanlar)

1. “Project settings” üzerinden yeni projeler için **default columns** tanımlanabilir; sütun isimleri virgülle ayrılır.
2. Varsayılan sütun örnekleri dokümantasyonda: **Backlog, Ready, Work in progress, Done** (Türkçe karşılıklarıyla).
3. **Categories proje bazlıdır**; ancak yeni projeler için otomatik oluşacak **default categories** listesi tanımlanabilir.
4. Opsiyonel kural: “Bir kullanıcının aynı anda yalnızca bir subtask alması” ayarı etkinleştirilebilir.

E) Remove Project (Projeyi Kaldırma/Silme)

1. Bir projeyi kaldırmak için **Project Manager** veya **Administrator** olmak gerekir.
2. Proje kaldırma işlemi, projeye ait **tüm task’ları da kaldırır** (data loss riski).

5.1.4 Business Rules (Bu Modülden Çıkan Kurallar)

1. Sistem birden fazla proje yönetimini destekler.
2. Proje tipi **Team** veya **Private** olabilir.
3. Private project’te kullanıcı yönetimi yoktur; herkes private project oluşturabilir.
4. Team project oluşturma yetkisi dokümantasyonda iki farklı şekilde ifade edildiği için uygulama davranışı doğrulanmalıdır.
5. Proje “private” yapıldığında mevcut kullanıcıların erişimi otomatik kaldırılmaz; erişim listesi manuel yönetilmelidir.
6. Proje görünümleri **Board/Calendar/List/Gantt** olup, her view filter/search sonucunu yansıtır.
7. Projeyi kaldırma, proje altındaki tüm task’ların da silinmesine neden olur.
8. Yeni proje oluştururken bazı proje özellikleri başka projeden çoğaltılabilir (permissions/actions/swimlanes/categories/tasks).

5.1.5 QA / Validation Notes (Test'e Dönüşecek Kontroller)

1. **Authorization check:** Team project oluşturma yetkisi (Administrator vs Manager vs Project Manager) gerçek sistem davranışıyla netleştirilecek.
2. **Data integrity:** Remove project sonrası tüm task'ların silindiği doğrulanacak (UI + mümkünse API).
3. **Visibility/Access:** Private ↔ team dönüşümü sonrası kullanıcı erişim davranışı ve permission modeli doğrulanacak.
4. **Defaults:** Yeni projede default columns/categories uygulandı mı (virgül ayrımı dahil) kontrol edilecek.
5. **Views consistency:** Board/Calendar/List/Gantt filtre sonuçları tutarlı mı, drag & drop çalışıyor mu (board).

5.2 Board Module (Pano / Columns / Swimlanes)

5.2.1 Overview

Kanboard'da iş akışının ana ekranı **Board (Pano)** görünümüdür. Task'lar **column (sütunlar)** içinde yer alır; opsiyonel olarak **swimlane (kulvar)** ile yatayda gruplandırılabilir. Board görünümü task'ların görsel akış yönetimini (drag & drop) sağlar.

5.2.2 Board View (Pano Görünümü) – Core Capabilities

A) Drag & Drop (Sütunlar arası taşıma)

1. Board görünümünde task'lar sütunlar arasında **drag & drop** ile taşınabilir.
2. Board'a geçiş için klavye kısayolu: **"v b"**.

B) "Recently moved" highlight (Görev öne çıkarma / gölge)

1. Yakın zamanda taşınan görevlerin çevresinde gölge (highlight) görülebilir.
2. Bu davranış "Pano Ayarları"ndan yönetilir; varsayılan süre **2 gün = 172800 saniye**, kapatmak için **0** verilir.

C) WIP / Task Limit Visual Indicator (Görev sınırı)

1. Bir sütun için **görev sınırına (task limit)** ulaşıldığında sütun arka planı **kırmızı** olur; bu, aynı anda çok fazla işin "in progress" olduğunu gösterir.

D) Column Visibility (Sütun göster/gizle)

1. Board üzerinde sütunlar pratik şekilde **gizlenebilir/gösterilebilir**.

2. Gizlemek için sütun dropdown menüsünden **"Bu sütunu gizle"**, geri getirmek için **"+ (artı) simgesi"** kullanılır.

E) Compact mode & Horizontal scrolling (Kompakt mod / yatay kaydırma)

1. Pano ekrana sığmadığında **yatay kaydırma çubuğu** görünür; tüm sütunları aynı anda görmek için **compact view** kullanılabilir.

2. Geçiş: klavye kısayolu **"c"** veya sol üst menü.

F) Collapsed vs Expanded Cards (Daraltılmış / Genişletilmiş kart görünümü)

1. Board'daki task kartları **collapsed** veya **expanded** modda gösterilebilir.

2. Geçiş: klavye kısayolu **"s"** veya sol menü.

3. Collapsed modda: atanan kişi varsa **baş harfler** görünür; uzun başlık için hover ile tooltip gösterilebilir.

5.2.3 Columns (Sütunlar) – Defaults & Rules

A) Default columns (Yeni projeler için varsayılan sütunlar)

1. Yeni projeler için varsayılan sütun adları "Project settings" üzerinden değiştirilebilir.

2. Sütun adları **virgülle ayrılır**.

3. Dokümanda verilen varsayılan örnek: **Backlog (Bekleme Listesi), Ready (Hazır), Work in progress (İşlemde), Done (Tamamlandı)**.

B) Column task limit (WIP limit) – BA doğrulama notu

1. Dokümana göre limit dolduğunda sütun arka planı kırmızı olur.

2. **Not:** Limit değerinin nereden tanımlandığı bu TR sayfada detaylı anlatılmıyor; sistemde ilgili ekran/ayar bulunup BA'da netleştirilecek (UI + mümkünse API/DB).

5.2.4 Swimlanes (Kulvarlar) – Yönetim & Davranış

A) Definition & usage

1. Swimlane'ler board üzerinde **yatay ayrımlardır**; ör. sürüm bazlı ayırmak, ürün/ekip bazlı bölmek için kullanılır.

B) Board behavior

1. Swimlane daraltma: soldaki ikon ile swimlane'ler daraltılabilir.

2. **Default swimlane** her zaman üstte gösterilir.

3. Task'lar swimlane'ler arasında **drag & drop** ile taşınabilir.

C) Manage swimlanes

1. Tüm projeler varsayılan bir swimlane ile gelir.
2. Swimlane ayarları **Project configuration > Swimlanes** bölümünden yapılır: yeni swimlane ekleme, rename, disable, sıralama değişimi.
3. Default swimlane üstte kalır ama **gizlenebilir**; inactive swimlane board'da görünmez.
4. Bir swimlane kaldırılırsa, o swimlane'e atanmış task'lar **silinmez**, **default swimlane'e taşınır**.

5.2.5 Board Settings (Pano Ayarları) – Non-Functional / Behavior

1. "Task highlighting" süresi saniye bazında ayarlanır (0=disable, default 172800).
2. Public shared board: sayfa varsayılan olarak **60 saniyede** bir yenilenir.
3. Private board: board açıkken değişiklik kontrolü varsayılan olarak **10 saniyede** bir yapılır (Ajax polling).

5.2.6 Business Rules (Özet Kurallar)

1. Board view task'ların column'lar arasında drag & drop taşınmasını destekler; kısayol: **v b**.
2. Yakın zamanda taşınan task'lar gölge ile vurgulanabilir; süre "Pano Ayarları"ndan yönetilir (0=kapalı, default 172800 sn).
3. Column task limit dolduğunda sütun arka planı kırmızı olur.
4. Column'lar board üzerinde hide/show edilebilir (dropdown + "+" simgesi).
5. Board compact view / yatay kaydırma geçişi "c" ile yapılabilir.
6. Task card görünümü collapsed/expanded arasında "s" ile değiştirilebilir.
7. Swimlane'ler yatay ayrımdır; default swimlane üstte; task'lar swimlane'ler arasında drag & drop taşınır.
8. Swimlane kaldırma task silmez; task'lar default swimlane'e taşınır.
9. Yeni projeler için default column isimleri virgülle ayrılarak tanımlanır.

5.2.7 QA / Validation Notes (Test'e dönüşecek kontroller)

1. **Drag & drop:** Column değişimi sonrası UI'da yer değişimi + "recently moved shadow" davranışı (süre/disable) doğrulanacak.
2. **Task limit:** Limit değerinin nereden set edildiği bulunup BA'da netleştirilecek; limite ulaşınca kırmızı arka plan, limit altı/üstü boundary testleri yapılacak.

3. **Show/Hide columns:** Hide edilen sütun geri getirilebiliyor mu (+ icon), davranış kullanıcı bazlı mı proje bazlı mı (doküman söylemiyor → sistemde gözlemlenecek).

4. **Compact/Collapsed toggles:** "c" ve "s" geçişleri çalışıyor mu; görünüm tercihi persist ediyor mu (doküman söylemiyor → gözlemlenecek).

5. **Swimlane removal behavior:** Swimlane kaldırılınca task'lar default swimlane'e taşınıyor mu; hiçbir task kaybolmuyor mu.

6. **Refresh intervals:** Public shared board 60sn refresh, private board 10sn polling davranışı doğrulanacak.

5.3 Görevleri Çoğaltma ve Taşıma (Duplicate / Move Tasks)

5.3.1 Amaç

Mevcut bir **Task**'ın (görev) aynı proje içinde veya başka bir projede yeniden üretilmesi (**Duplicate**) ya da başka bir projeye aktarılması (**Move**) sürecini tanımlamak.

5.3.2 Aktörler ve Roller

1. **Project Member**: Kendi erişimi olan projelerde görev çoğaltma/taşıma işlemlerini başlatır.

2. **Project Manager / Admin**: (Varsa) proje yapılandırmaları, izinler ve hedef projedeki alanların (swimlane/column/category vb.) yönetiminden sorumludur. *(Yetki matrisi 4.x/İzinler bölümünüzle birlikte doğrulanmalı.)*

5.3.3 Ön Koşullar

1. Kaynak görev (**source task**) mevcut olmalı ve kullanıcı görev detay ekranına erişebilmelidir.

2. Hedef projeye çoğaltma/taşıma için kullanıcı **hedef projeye üye** olmalıdır (seçim listesinde yalnızca üye olunan projeler görünür).

5.3.4 Fonksiyonel Gereksinimler ve İş Akışları

FR-5.3.1 Aynı Projeye Çoğalt (Duplicate within same project)

Akış:

1. Kullanıcı görev detayına (**task view**) gider.

2. Sol menüden **Çoğalt (Duplicate)** seçeneğini seçer.

3. Sistem, orijinalle aynı özelliklere sahip yeni bir görev oluşturur.

Beklenen Sonuç:

1. Yeni görev aynı projede oluşur (orijinalden kopyalanan alanlar 5.3.5'te).

FR-5.3.2 Başka Bir Projeye Çoğalt (Duplicate to another project)

Akış:

1. Kullanıcı görev detayına gider.
2. **Başka bir projeye çoğalt (Duplicate to another project)** seçeneğini seçer.
3. Sistem, açılır listede **yalnızca üye olunan projeleri** gösterir.
4. Sistem, kaynak ve hedef proje arasında ortak olmayan özellikler için kullanıcıdan hedef değerleri ister.
5. Kullanıcı en az şu alanları hedef proje için seçer:
    1. **Swimlane (Hedef kulvar)**
    2. **Column (Sütun)**
    3. **Category (Kategori)**
    4. **Assignee (Atanan kişi)**

Beklenen Sonuç:

1. Yeni görev hedef projede, kullanıcı tarafından seçilen alanlarla oluşturulur.

FR-5.3.3 Başka Bir Projeye Taşı (Move to another project)

Akış:

1. Kullanıcı görev detayına gider.
2. **Başka bir projeye taşı (Move to another project)** seçeneğini seçer.
3. Sistem, taşıma işlemini "başka projeye çoğaltma" ile aynı mantıkla yürütür; kullanıcı hedef projede görevin yeni özelliklerini seçer.

Beklenen Sonuç:

1. Görev hedef projeye aktarılmış olur; hedef özellikler kullanıcı seçimine göre set edilir.

5.3.5 Çoğaltılan Alanlar (Duplicated Properties)

Çoğaltma sırasında kopyalanan alanlar:

1. title

2. description
3. date_due
4. color_id
5. project_id
6. column_id
7. owner_id
8. score
9. category_id
10. time_estimated
11. swimlane_id
12. recurrence_status
13. recurrence_trigger
14. recurrence_factor
15. recurrence_timeframe
16. recurrence_basedate

5.3.6 İş Kuralları / Validasyonlar

1. Hedef projeye çoğaltmada, kullanıcıya yalnızca **üye olduğu projeler** listelenmelidir.
2. Kaynak ve hedef projede ortak olmayan özellikler varsa sistem kullanıcıdan hedef seçimleri istemelidir (swimlane/column/category/assignee).

5.3.7 Kabul Kriterleri (Acceptance Criteria)

1. **AC-5.3.1:** Aynı projeye çoğaltma yapıldığında yeni görev oluşturulur ve 5.3.5'teki alanlar kopyalanır.
2. **AC-5.3.2:** Başka projeye çoğaltmada kullanıcı yalnızca üye olduğu projeleri görür; hedef alan seçimleri yapılmadan işlem tamamlanamaz.
3. **AC-5.3.3:** Başka projeye taşımada, kullanıcı hedef görev özelliklerini seçer ve görev hedef projeye aktarılır.

5.3.8 Test Kapsam Notları (QA için – BA içi)

1. **UI (Manual/Automation):**

    1. Task view → Duplicate / Duplicate to another project / Move to another project akışları.
    2. Hedef proje dropdown'unda "üye olunan projeler" kuralı.
    3. Hedef alan seçimleri (swimlane/column/category/assignee) zorunluluk ve doğru set edilmesi.

2. **API:**
    1. (API endpoint'leri ayrı bölümde netleştirilecek) Duplicate/Move sonrası alanların gerçekten kopyalandığını/taşındığını doğrulayan kontroller.

5.4 User & Access Management Module (Kullanıcı / Grup / Yetkilendirme)

5.4.1 Overview

Bu modül; kullanıcıların sisteme eklenmesi/çıkarılması, grupların yönetimi, rol (role) ve izin (permission) modelinin proje bazında uygulanması ile kimlik doğrulama seçeneklerini kapsar. Yetkilendirme hem **application level roles** hem de **project level roles** üzerinden çalışır.

5.4.2 User Management (Kullanıcı Yönetimi)

A) Create User (Yeni kullanıcı ekleme)

1. Yeni kullanıcı eklemek için **Administrator (Yönetici)** olmak gerekir.
2. Yeni kullanıcı ekleme akışı: Kullanıcı Yönetimi menüsü → "Yeni yerel kullanıcı" veya "Yeni uzak kullanıcı" seçimi → form doldur/kaydet.
3. **Local user** oluştururken zorunlu alanlar:
    1. username (benzersiz giriş kimliği)
    2. password (en az **6 karakter**)
4. **Remote user** oluştururken yalnızca username zorunludur.

B) Edit User (Kullanıcı düzenleme)

1. Normal kullanıcı yalnızca **kendi profilini** düzenleyebilir.
2. Herhangi bir kullanıcıyı düzenleyebilmek için **Administrator** olmak gerekir.

C) Remove User (Kullanıcı kaldırma)

1. "Kaldır" bağlantısı yalnızca **Administrator** ise görünür.
2. Bir kullanıcı kaldırıldığında, o kullanıcıya atanmış task'lar **unassigned** olur.

BA Business Rule (User Management):

1. User create/edit/remove işlemleri admin yetkisi gerektirir (kendi profilini düzenleme hariç).
2. Local user password policy: minimum 6 karakter.
3. Kullanıcı silinince görevler silinmez, sadece ataması kaldırılır (unassigned).

5.4.3 Group Management (Grup Yönetimi)

1. Her kullanıcı bir veya daha fazla gruba üye olabilir; grup "takım/organizasyon" gibi düşünülebilir.
2. Yeni grup oluşturma ve kullanıcıları gruba atama işlemleri **yalnızca Administrator** tarafından yapılabilir.
3. Gruplar "Kullanıcı Yönetimi > Tüm Grupları Görüntüle" ekranından yönetilir.
4. Proje yöneticisi, **Proje İzinleri (Project permissions)** sayfasından bir gruba proje erişim yetkisi verebilir.
5. "External identity" kavramı, harici grup sağlayıcıları (örn. LDAP) için kullanılır; Kanboard LDAP grup sağlayıcısı ile grupları senkronize edebilir.

BA Business Rule (Groups):

1. Group create/assign admin yetkisi gerektirir.
2. Grup bazlı proje erişimi project permissions üzerinden verilebilir.

5.4.4 Roles Model (Roller)

Kanboard iki seviyede rol modelini tanımlar:

A) Application Roles

Role	Açıklama
Administrator	Her şeye erişim
Manager	Ekip projeleri oluşturabilir, ancak uygulama ayarlarını değiştiremez
User	Sadece özel projeler oluşturabilir

B) Project Roles (Team projects için)

Role	Açıklama
Project Manager	Proje ayarlarını değiştirebilir; Gantt ve raporlara erişebilir
Project Member	Task oluşturabilir ve board'u kullanabilir

Role	Açıklama
Project Viewer	Board ve task'lara read-only erişim sağlar

5.4.5 Project Permissions (Proje İzinleri / Erişim Modeli)

1. Her proje birbirinden izoledir; projeye erişim, **proje sahibi tarafından** verilmelidir.
2. Her kullanıcıya ve her gruba projede ayrı rol atanabilir (Project Manager/Member/Viewer).
3. Rol atamaları "Proje Ayarları > İzinler" ekranından yapılır.
4. Yalnızca **Administrator** her şeye erişime sahiptir.
5. **Private projects** için permission tanımlanamaz.

BA Business Rule (Permissions):

1. Proje erişimi owner tarafından verilir; user/group bazlı rol atanır.
2. Private project'lerde permission management yoktur.

5.4.6 Custom Project Roles (Özel Proje Rolleri)

1. Özel proje rolleri, proje üyelerine belirli kısıtlar uygulamak için kullanılır ve **her proje için ayrı** tanımlanır.
2. Custom role, **Project Member** rolünden devralır; üzerine kısıtlamalar eklenir.

Mevcut kısıtlama örnekleri (dokümana göre):

1. Proje kısıtlamaları: task creation yasak, task close/open yasak, task move yasak
2. Sütun kısıtlamaları: sadece belirli sütunda task creation izinli/engelli; belirli sütunda close/open izinli/engelli
3. Drag & drop kısıtları: sadece belirtilen sütunlar arasında taşıma

Atama:

1. Custom role ataması "İzinler" bölümünden kullanıcıya yapılır.

BA Business Rule (Custom Roles):

1. Custom roles project-specific ve restriction-based çalışır; authorization testlerinde "allowed/blocked" senaryoları gerekir.

5.4.7 User Types (Kullanıcı Tipleri)

Kanboard iki kullanıcı tipi tanımlar:

1. **Local User:** şifre Kanboard veritabanında saklanır

2. **Remote User:** kimlik bilgileri başka bir sistem tarafından yönetilir (örn. LDAP, reverse proxy)

5.4.8 Two-Factor Authentication (2FA)

1. Her kullanıcı **two-factor authentication (2FA)** özelliğini aktif edebilir.

2. Başarılı login sonrası kullanıcıdan 6 karakterlik **one-time code** istenir.

Detaylı 2FA/TOTP gereksinimleri ve kabul kriterleri için bkz. 5.20.

5.4.9 Application Configuration (Uygulama Ayarları) – Yönetimsel Kontrol

1. Uygulama ayarları sayfasındaki parametreleri **yalnızca Administrator** değiştirebilir.

2. "Application URL" e-posta bildirimlerinde kullanılır (mail footer'da task linki).

3. "Language" ayarı uygulama dilini belirler ve tüm kullanıcılar için geçerli olur.

5.4.10 QA / Validation Notes (Test'e dönüşecek kontroller)

1. **User CRUD authorization:** Admin olmayan kullanıcı "user create/remove" yapamamalı; normal user sadece kendi profilini edit edebilmeli.

2. **Password policy:** local user password minimum length (6) doğrulanmalı.

3. **User removal impact:** kullanıcı silinince task assignment "unassigned" olmalı (data kaybı olmamalı).

4. **Group management:** admin dışında grup create/assign engellenmeli; project permissions üzerinden group access verilebilmeli.

5. **Project permissions:** private project'te permission ekran davranışı/limitleri doğrulanmalı.

6. **Custom roles restrictions:** restriction tanımı sonrası UI'da "blocked action" gerçekten engelleniyor mu (task move/close/create vb.).

7. **2FA:** enable sonrası login akışında 6 haneli code zorunluluğu doğrulanmalı.

5.5 Notifications & Subscriptions Module (Bildirimler / Mention / RSS-Atom / iCal / Email-to-Task)

Not: Tekrarlı içeriği önlemek için bu konu başlığı ayrıştırılmıştır; detaylar aşağıdaki bölümlerdedir:

- 5.17 RSS / Atom Abonelikleri

- 5.18 Takvim Senkronizasyonu (iCalendar / iCal)

- 5.19 E-posta ile Görevler Oluşturma (Create Tasks by Email)

- 5.21 Bildirimler (Notifications) ve Kullanıcı Bahsetmeleri (User Mentions)

5.6 Automatic Actions Module (Otomatik İşlemler)

5.6.1 Overview

Kanboard, kullanıcı etkileşimini azaltmak için **Automatic Actions** (otomatik işlemler) mekanizmasını destekler.
Bir otomatik işlem; belirli bir **event** gerçekleştiğinde, önceden tanımlanmış bir **action**'ın belirli **parametreler** ile otomatik çalıştırılmasıdır.

5.6.2 Automatic Action Model (Tanım / Bileşenler)

Dokümantasyona göre her otomatik işlem 3 parçadan oluşur:

1. **Event (Etkinlik):** Dinlenecek olay (ör. "task başka kolona taşındı")
2. **Action (İşlem):** Olay gerçekleşince yapılacak işlem (ör. "task'ı kapat")
3. **Parameters (Parametreler):** Action'ın çalışması için gereken hedef değerler (ör. kolon adı/id, kullanıcı, proje vb.)

Scope (Kapsam): Otomatik işlemler **proje bazlıdır**; her projenin kendine ait otomatik işlem seti vardır.
UI Location: Proje giriş/konfigürasyon panelinde "Otomatik İşlemler" bağlantısı altında yapılandırılır.

5.6.3 Add New Automatic Action (UI ile Yeni Eylem Ekleme)

Yeni otomatik işlem ekleme adımları:

1. **Yeni bir otomatik işlem ekle** bağlantısına tıkla
2. **Action** seç
3. **Event** seç
4. Action'a ait **Parameters** alanlarını doldur ve kaydet

5.6.4 Available Actions Inventory (Kullanılabilir İşlemler – Özet Envanter)

Dokümantasyon "kullanılabilir işlemler" olarak aşağıdaki türleri örnekler (tam liste UI'da görünebilir):

1. Task'a otomatik **category** atama (renk/etiket/bağlantı bazlı)
2. Task'a otomatik **color** atama (kategori/kolon/kullanıcı bazlı)

3. Task'ı otomatik **assign** etme (specific user / current user / external username)

4. Task **close** / **open** işlemleri (genel veya belirli kolonda)

5. Task'ı başka projeye **duplicate** veya **move** etme

6. Task taşınırken otomatik **comment/log** ekleme

7. Task **start date**'ini otomatik güncelleme

8. External provider'dan task/comment oluşturma

9. Task'ı e-posta ile gönderme

BA Notu: Bu liste, 5.6 kapsamının "neleri test edeceğiz" envanteridir. Ayrıntı parametre setleri UI + API ile doğrulanacaktır.

5.6.5 Documented Example Configurations (Dokümandaki Örnek Senaryolar)

Aşağıdaki örnekler Kanboard dokümantasyonunda **konfigürasyon reçetesi** olarak verilir ve test case türetmek için çok uygundur:

EX-1: "Done/Bitti" kolonuna taşınınca task otomatik kapansın

1. Action: "Bir görevi belirli bir sütunda kapatın"

2. Event: "Görevi başka bir kolona taşıyın"

3. Params: Column = "Bitti"

EX-2: "Doğrulanacak" kolonuna taşınınca task belirli kullanıcıya atansın (örn. Bob)

1. Action: "Görevi belirli bir kullanıcıya atayın"

2. Event: "Görevi başka bir kolona taşıyın"

3. Params: Column = "Doğrulanacak", User = "Bob"

EX-3: "Çalışma Sürüyor" kolonuna taşınınca task'ı işlemi yapan kişiye ata (current user)

1. Action: "Görevi, kolon değiştiğinde işlemi yapan kişiye atayın"

2. Event: "Görevi başka bir kolona taşıyın"

3. Params: Column = "Çalışma Sürüyor"

EX-4: Task kapatılınca başka projeye kopyala (duplicate)

1. Action: "Görevi başka bir projeye çoğaltın"

2. Event: "Görevi kapatma"

3. Params: Column = "Doğrulanmış", Project = hedef proje

EX-5: Task son kolona taşınınca başka projeye taşı (move)

1. Action: “Görevi başka bir projeye taşıyın”
2. Event: “Görevi başka bir kolona taşıyın”
3. Params: Column = “Doğrulanmış”, Project = hedef proje

EX-6: Kolona taşınınca start date otomatik set edilsin

1. Action: “Başlangıç tarihini otomatik olarak güncelleyin”
2. Event: “Görevi başka bir kolona taşıyın”
3. Params: Column = “Çalışma Sürüyor”

5.6.6 Business Rules (Özet Kurallar)

1. Otomatik işlemler proje bazlıdır; her projenin action seti ayrıdır.
2. Her otomatik işlem; **event + action + parameters** bileşenlerinden oluşur.
3. Örneklerdeki gibi action’lar, seçilen event gerçekleştiğinde tanımlı parametrelerle çalışır (örn. kolona taşınma → kapatma/atama/start date set).

5.6.7 API Support (Automatic Actions – JSON-RPC)

Automatic Actions, API üzerinden de listelenebilir/konfigüre edilebilir (test ve otomasyon açısından kritik):

1. getAvailableActions (kullanılabilir action listesi)
2. getAvailableActionEvents (kullanılabilir event listesi)
3. getCompatibleActionEvents (seçilen action için uyumlu event’ler)
4. getActions (project_id için mevcut action konfigürasyonları)
5. createAction (yeni action oluşturma: project_id + event_name + action_name + params)
6. removeAction (action_id ile silme)

BA Notu: API ile “konfigürasyon yönetimi” yapılabildiği için, UI ve API sonuçlarının tutarlılığı test kapsamına alınmalıdır (UI’da ekle → API’de görünür mü, API’de ekle → UI’da görünür mü).

5.6.8 QA / Validation Notes (Test’e Dönüşecek Kontroller)

A) UI Functional

1. Action ekleme akışında (Action → Event → Params) zorunlu alan validasyonları.

2. EX-1/2/3 gibi “kolona taşındı” event’inde doğru action tetikleniyor mu (close/assign/start date).

3. Duplicate/Move to another project örneklerinde hedef proje/kolon parametreleri doğru uygulanıyor mu.

B) Negative / Security

1. Kullanıcı yetkisi olmayan projede action konfigüre edebiliyor mu? (RBAC kontrol)

2. Parametrelerde “olmayan kolon/kullanıcı/proje” seçimi mümkün mü? mümkünse hata davranışı ne? (UI ve API)

C) API Validation

1. getAvailableActions / getAvailableActionEvents çıktıları boş/false dönme durumları.

2. createAction ile eklenen action’ın getActions çıktısında görünmesi.

3. removeAction sonrası UI/API tutarlılığı.

5.7 Kanboard API Layer (JSON-RPC API)

5.7.1 Overview

Sistem, dış uygulamalarla entegrasyon için **JSON-RPC v2** tabanlı API sağlar. API çağrıları **HTTP POST** ile yapılır ve **batch requests** (tek HTTP request içinde birden fazla çağrı) desteklenir.

5.7.2 API Access Types (Erişim Tipleri)

API erişimi iki tiptir:

A) Application API

1. API’ye “jsonrpc” kullanıcısı ve Settings sayfasındaki token ile erişilir.

2. Tüm prosedürlere erişim hedeflenir; **permission check yapılmaz** ve sunucuda **user session oluşturulmaz**.

3. “My…” ile başlayan prosedürlere erişim yoktur (örn. getMe, getMyProjects...).

B) User API

1. Normal kullanıcı kimlik bilgileri ile erişilir; parola yerine **personal access token** kullanılabilir.
2. Her prosedürde **application roles + project permissions** kontrol edilir; sunucuda **user session oluşturulur**.

5.7.3 Endpoint & Transport

1. API endpoint: https://YOUR_SERVER/jsonrpc.php
2. Protocol: JSON-RPC v2, transport: HTTP POST
3. Batch requests: desteklenir (tek request içinde çoklu çağrı).

5.7.4 Security Requirements (Dokümana dayalı)

1. İletişimde **HTTPS** kullanılması önerilir.
2. User API tarafında art arda **3 başarısız kimlik doğrulama** sonrasında kullanıcının hesabını login formu üzerinden açması gerekir.
3. **2FA (two-factor authentication)** aktif kullanıcılar API tarafında **API keys** kullanmalıdır (v1.2.8+).

5.7.5 Authentication Methods

5.7.5.1 Default Method: HTTP Basic

HTTP Basic Authentication (RFC2617) kullanılır.

Application credentials

1. Username: jsonrpc
2. Password: Settings sayfasındaki **API token**

User credentials

1. Username: kullanıcı adı
2. Password: kullanıcı parolası **veya** personal access token

5.7.5.2 Optional Method: Custom HTTP Header

Sunucu kısıtları nedeniyle Basic kullanılamıyorsa alternatif bir HTTP header ile kimlik doğrulama yapılabilir.

1. Header name: serbest (örn. X-API-Auth)
2. Header value: username:password Base64 encoded
3. Konfigürasyon: config.php içinde API_AUTHENTICATION_HEADER tanımlanır.

5.7.6 Error Handling (HTTP Seviyesi)

1. Kimlik doğrulama hatası (credentials wrong): **401 Not Authorized**
2. Yetkilendirme hatası (user allowed değil): **403 Forbidden**

5.7.7 JSON-RPC Request/Response Format (Template)

Request Template

```
{
 "jsonrpc": "2.0",
 "method": "<methodName>",
 "id": 1,
 "params": {}
}
```

JSON-RPC v2 kullanımı, POST zorunluluğu ve batch desteği dokümanda belirtilmiştir.

Not: JSON-RPC "error object" alanlarının (code/message/data) kesin şeması, sistemde gerçek hata örnekleriyle doğrulanarak "Appendix: Error Schema" bölümüne eklenecektir (Validation Item).

5.7.8 API Procedures (Procedure Group Inventory)

Dokümantasyon API prosedürlerini aşağıdaki gruplarda listeler:

1. Application API Procedures
2. Automatic Actions API Procedures
3. Board API Procedures
4. Category API Procedures
5. Column API Procedures
6. Comment API Procedures
7. Current User API Procedures
8. External Task Link API Procedures
9. Group API Procedures
10. Group Member API Procedures
11. Internal Task Link API Procedures
12. Link API Procedures

13. Project API Procedures
14. Project File API Procedures
15. Project Metadata API Procedures
16. Project Permission API Procedures
17. Subtask API Procedures
18. Subtask Time Tracking API Procedures
19. Swimlane API Procedures
20. Tags API Procedures
21. Task API Procedures
22. Task File API Procedures
23. Task Metadata API Procedures
24. User API Procedures

5.7.9 Core API Capabilities (Kritik Metot Örnekleri)

Bu alt bölüm, test kapsamı için "yük taşıyan" metot örneklerini listeler (tam liste ilgili procedure sayfalarındadır).

A) Project API (örnekler)

1. createProject (project create)
2. removeProject: success true, failure false
3. enableProjectPublicAccess / disableProjectPublicAccess gibi public access prosedürleri

B) Task API (örnekler)

1. createTask (title + project_id required)
2. getTask: failure null
3. closeTask / openTask: success true, failure false
4. moveTaskPosition (aynı board içinde column/position/swimlane hareketi)
5. moveTaskToProject ve duplicateTaskToProject (projeler arası taşıma/çoğaltma)

C) Automatic Actions API (örnekler)

1. getAvailableActions, getAvailableActionEvents, getCompatibleActionEvents, getActions

D) Project Permission API (örnekler)

1. addProjectUser / addProjectGroup / changeProjectUserRole / removeProjectUser

E) Tags API (örnek)

1. getTagsByProject (failure false|null)

5.7.10 Validation Approach (API Test Kapsamı – BA Seviyesi)

API doğrulaması aşağıdaki başlıklar altında planlanacaktır. (Bu bölüm "plan/approach" tanımıdır.)

1. **Authentication Validation**

1. Application credentials ile erişim
2. User credentials (password / personal access token) ile erişim
3. Custom header ile erişim (ortam uygunsa)

1. **Authorization (RBAC) Validation**

1. User API çağrılarında role & project permissions kontrolü (allowed / denied)
2. Denied durumunda 403 doğrulaması

1. **Functional Contract Validation (Prosedür Sözleşmesi)**

1. Required param missing
2. Type mismatch (string/int vb.)
3. Invalid ID / not found
4. "Result on failure" davranışlarının procedure sayfalarına göre doğrulanması (örn. removeProject failure false, getTask failure null).

1. **Batch Requests Validation**

1. Batch request içinde çoklu çağrının doğru id ile eşleşen response üretmesi (order/id mapping).

1. **UI–API Consistency Validation**

1. UI üzerinden create/update yapılan kayıtların API tarafında doğrulanması ve ters yönde de tutarlılık kontrolü (özellikle Project/Task/Permissions/Automatic Actions).

5.7.11 Traceability (BA 5.x Modülleri ↔ API Procedure Groups)

1. Projects module → Project API + Project Permission API + Project Metadata API
2. Board/Columns/Swimlanes module → Board API + Column API + Swimlane API
3. Tasks/Subtasks/Time module → Task API + Subtask API + Subtask Time Tracking API + Task File/Metadata API

4. Automatic Actions module → Automatic Actions API procedures

5.7.12 API Smoke Suite (Minimum) – Önerilen Doğrulama Seti

Aşağıdaki set, ortamın "API temel çalışırlığı" için minimum doğrulama olarak kullanılabilir (procedure detayları ilgili sayfalara göre uygulanır):

1. **Auth sanity**

1. getAllProjects (basic connectivity)

1. **Project lifecycle**

1. createProject → removeProject

1. **Task lifecycle**

1. createTask → moveTaskPosition → closeTask → openTask → removeTask

1. **Permissions sanity (User API ile)**

1. addProjectUser / changeProjectUserRole / removeProjectUser

1. **Automatic Actions discovery**

1. getAvailableActions + getAvailableActionEvents

5.7.13 API Test Coverage Matrix (Procedure Group Bazlı)

Bu tablo, API testlerinin prosedür gruplarına göre minimum kapsamını ve test tiplerini gösterir. Bu proje bir öğrencilik/tek kişi çalışması olduğundan; SMTP, inbound email ve cronjob gibi altyapı gerektiren maddeler ortam yoksa 'Not Executed (Environment Constraint)' olarak raporlanacaktır.

Procedure Group	Örnek Metotlar	Auth	Functional	Negative /RBAC	Batch/Consistency
Project API	Project lifecycle (create / remove)	✓	✓	✓	UI–API consistency
Task API	Task lifecycle (create / move / close)	✓	✓	✓	UI–API consistency
Board/Column/Swimlane	Board structure (columns / swimlanes)	✓	✓	✓	Consistency
Project Permission API	Permissions (add / change / remove user)	✓ (User API)	✓	✓ (RBAC)	—
Subtask &	Subtasks + time tracking	✓	✓	✓	Consiste

Time Tracking					ncy
Tags API	Tags (set / get)	✓	✓	✓	—
Internal Task Link API	Internal links (create / remove)	✓	✓	✓	—
Task File / Project File	Files (upload / remove)	✓	✓	✓ (size/invalid)	—
Automatic Actions API	Automatic actions (list / create)	✓	✓	✓	—
Current User (My…)	Current user (getMe / getMyProjects)	✓	✓	✓	—

Kabul kriteri: Her procedure group için en az 1 pozitif (success) ve 1 negatif (failure) örnek çağrı çalıştırılmalı; RBAC doğrulaması yalnız User API ile yapılmalıdır.

5.8 Authorization & Roles (Yetkilendirme ve Roller)

5.8.1 Purpose

Bu bölüm; Kanboard'daki **application roles**, **project roles** ve **custom project roles** kapsamında, temel operasyonlar için beklenen yetkilendirme davranışlarını (authorization expectations) tanımlar.

5.8.2 Role Definitions

5.8.2.1 Application Roles (Uygulama Rolleri)

Kanboard'da her kullanıcı aşağıdaki application role'lerden birine sahiptir:

1. **Yönetici (Administrator):** Her şeye erişim.
2. **Müdür (Manager):** Ekip projeleri oluşturabilir; ancak uygulama ayarlarını değiştiremez.
3. **Kullanıcı (User):** Sadece özel projeler oluşturabilir.

Uygulama ayarları (Application settings) yalnızca yöneticiler tarafından değiştirilebilir.

5.8.2.2 Project Roles (Proje Rolleri – Team Project)

Her ekip projesinde (team project) her kullanıcıya/gruba aşağıdaki proje rolleri atanabilir:

1. **Proje Müdürü (Project Manager):** Proje ayarlarını değiştirebilir; Gantt grafiğine ve raporlara erişebilir.
2. **Proje Üyesi (Project Member):** Görevler oluşturabilir ve panoyu kullanabilir.
3. **Proje Görüntüleyici (Project Viewer):** Pano ve görevlere salt okunur erişim sağlar.

Proje izinleri proje ayarları altında yönetilir; her kullanıcıya/gruba rol atanabilir.

5.8.2.3 Custom Project Roles (Özel Proje Rolleri)

Özel proje rolleri her proje için tanımlanır ve **Project Member rolünden devralır**; belirli kısıtlama setleri uygulanabilir.

Mevcut kısıtlama türleri (dokümantasyon):

1. **Proje kısıtlamaları (Project restrictions):**
    1. Task creation not permitted
    2. Closing/opening a task not permitted
    3. Moving a task not permitted
2. **Sütun kısıtlamaları (Column restrictions):**
    1. Task creation allowed only for a specific column / blocked for a specific column
    2. Closing/opening allowed only for a specific column / blocked for a specific column
3. **Belirli sütunlar arasında taşıma (Move only between specified columns)**

5.8.3 Authorization Matrix (Yetki Matrisi)

5.8.3.1 Application-Level Authorization Matrix

Operation (Uygulama Seviyesi)	**Administrator**	**Manager**	**User**	**Notes**
Application settings değişikliği	Allowed	Not allowed	Not allowed	"Uygulama ayarlarını yalnızca yöneticiler değiştirebilir"; Manager için "uygulama ayarlarını değiştiremez".
User Management: yeni kullanıcı ekleme	Allowed	Not specified	Not specified	"Yeni bir kullanıcı eklemek için yönetici olmalısınız."
User Management: herhangi bir kullanıcıyı düzenleme	Allowed	Not specified	Not allowed	"Normal kullanıcı yalnızca kendi profilini değiştirebilir; herhangi bir kullanıcıyı düzenlemek için yönetici olmalı."
Group Management: grup oluşturma ve kullanıcı atama	Allowed	Not specified	Not specified	"Yalnızca yöneticiler yeni gruplar oluşturabilir ve kullanıcıları atayabilir."
Team project oluşturma	**Validation Item**	**Validation Item**	Not allowed	Dokümanda çelişen ifadeler mevcut (bkz. 5.8.5 Validation Items).
Private project oluşturma	Allowed	Allowed	Allowed	"Özel projeler herkes tarafından oluşturulabilir."

5.8.3.2 Project-Level Authorization Matrix (Team Projects)

Operation (Proje Seviyesi)	**Project Manager**	**Project Member**	**Project Viewer**	**Notes**
Proje ayarlarını değiştirme (project settings)	Allowed	Not specified	Not specified	Proje Müdürü "proje ayarlarını değiştirebilir."
Gantt chart + reports erişimi	Allowed	Not specified	Not specified	Proje Müdürü "Gantt grafiğine ve raporlara erişebilir."
Task oluşturma + board kullanımı	Not specified	Allowed	Not allowed	Project Member "Görevler oluşturabilir ve panoyu kullanabilir"; Project Viewer read-only.
Project permissions (rol atamaları) yönetimi	**Derived / Validation Item**	Not specified	Not specified	Rol atamaları "Proje Ayarları > İzinler" altında; hangi proje rolünün yönetebileceği açıkça yazmıyor. Beklenti: Project Manager (project settings yetkisi nedeniyle).
Custom project roles (özel rol) oluşturma/kısıtlama tanımı	**Derived / Validation Item**	Not specified	Not specified	Custom roles "Proje ayarları" altında yapılandırılır; erişim beklentisi Project Manager (project settings yetkisi nedeniyle).
Automatic actions yapılandırma	**Derived / Validation Item**	Not specified	Not specified	Automatic actions proje bazlıdır ve proje yapılandırma panelinden yönetilir; erişim rolü açık yazmıyor. Beklenti: Project Manager.
Proje kaldırma (remove project)	Allowed	Not allowed	Not allowed	"Bir projeyi kaldırmak için, projenin müdürü veya yönetici olmanız gerekir."
Gruplara erişim yetkisi verme (projeye grup authorize)	Allowed	Not specified	Not specified	"Her proje yöneticisi… proje izinleri sayfasından bir gruba erişim yetkisi verebilir."

5.8.3.3 Private Projects (Özel Projeler) – Authorization Notes

1. Özel projelerde **izin tanımlanamaz**.
2. Özel proje oluşturma bölümünde: projeye yalnızca **sahibi ve yöneticiler** erişebilir.

3. Projeyi özel/çok kullanıcılı projeye dönüştürme işlemi: “Yöneticiler ve proje yöneticileri, ‘Özel Proje’ onay kutusunu değiştirerek…”

5.8.4 Custom Project Roles – Functional Constraints (Testable Rules)

Özel proje rolü, Project Member’dan devralır ve kısıtlamalarla daraltılır.

Bu kapsamda test/doğrulama için kural seti:

1. **Task creation restriction**: Task oluşturma tamamen kapatılabilir veya yalnız belirli sütunda açılabilir/bloke edilebilir.

2. **Task status restriction**: Task close/open tamamen kapatılabilir veya yalnız belirli sütunlarda açılabilir/bloke edilebilir.

3. **Task move restriction**: Task move tamamen kapatılabilir veya yalnız belirtilen sütunlar arası geçişe izin verilecek şekilde sınırlandırılabilir.

4. Dokümantasyonda “kural kombinasyonunun önemli olduğu” açıkça belirtilmiştir; bu nedenle kısıtlama kombinasyonları birlikte test edilmelidir.

5.8.5 Validation Items (Doküman Çelişkisi / Doğrulanacaklar)

1. **Team project oluşturma yetkisi – doküman çelişkisi**

    1. “Proje Tipleri” sayfası: yalnızca “yöneticiler ve uygulama yöneticileri (Manager)” team project oluşturabilir.

    2. “Projeleri Oluşturma” sayfası: yalnızca “yöneticiler ve proje yöneticileri (Project Manager)” team project oluşturabilir.

    3. “Kullanıcı Rolleri” sayfası: Manager team project oluşturabilir; User yalnız private project oluşturabilir.
    Doğrulama yaklaşımı: UI üzerinden role bazında proje oluşturma denemeleri + API üzerinden aynı deneme (User API ile) yapılacaktır.

2. **Project permissions / automatic actions / custom roles UI erişimi**
    Bu sayfaların hangi proje rolü tarafından yönetilebildiği dokümantasyonda açıkça “role requirement” olarak belirtilmediği için, proje rolleri bazında UI erişimi doğrulanacaktır.

5.9 Open Items & Appendices (Açık Maddeler ve Ekler)

Not: Açık maddeler ve doğrulama (validation) kalemleri tek yerde toplanmıştır; bkz. 5.29. Ekler ise 5.30 ve sonrası başlıklardadır.

5.10 Alt Görevler (Subtasks)

5.10.1 Tanım ve Amaç

Alt görevler, bir görevin işini daha küçük parçalara bölmek için kullanılır.

5.10.2 Alt Görev Özellikleri (Functional Attributes)

Her alt görev için:

1. Bir **proje üyesine (project member)** atanabilir.
2. 3 durum (status) içerir: **Yapılacak (To Do)**, **Devam ediyor (In Progress)**, **Bitti (Done)**.
3. Zaman bilgileri içerir: **geçen süre (time spent / elapsed)** ve **tahmini süre (time estimated)**.
4. Pozisyona göre sıralanır (ordering by position).

5.10.3 UI – Alt Görev Oluşturma

Alt görev oluşturma akışı:

1. Görev görünümünde (task view), sol menüden **"Bir alt görev ekle"** seçeneği ile ekleme yapılır.
2. Alternatif olarak yalnızca başlık girilerek hızlı ekleme yapılabilir.

5.10.4 UI – Alt Görev Durum Yönetimi

1. Alt görev başlığına tıklanarak durum değiştirilebilir; başlıktan önceki ikon duruma göre güncellenir.
2. **Kural:** Görev kapatıldığında (task closed), tüm alt görevler **Bitti** durumuna geçirilir.

5.10.5 Alt Görev Zamanlayıcısı (Subtask Timer) – Davranış

Alt görev zamanlayıcısı (timer) ile ilgili kurallar:

1. Zamanlayıcı istenildiğinde başlatılabilir/durdurulabilir (her zaman).
2. Zamanlayıcı, alt görev için geçen süreyi otomatik kaydeder; "geçen süre" alanı manuel düzenlenebilir.
3. Hesaplanan süre en yakın çeyreğe (quarter) yuvarlanır; bilgi ayrı bir tabloda kaydedilir. Not: Dokümantasyonda bu davranışın Kanboard < 1.0.32 için geçerli olabileceği belirtilir; kullanılan sürümde doğrulanacaktır.
4. Geçen süre ve tahmini süre değerleri, alt görevlerin toplamına göre otomatik güncellenir.

5.10.6 API Traceability (BA İzlenebilirlik)

Alt görev fonksiyonları için API tarafında ilgili procedure grupları bulunmaktadır:

1. **Subtask API Procedures**
2. **Subtask Time Tracking API Procedures**

5.10.7 Validation Items (Doğrulama Maddeleri)

1. Alt görev atamasının yalnızca proje üyelerine yapılabildiği doğrulanacaktır.
2. Parent task kapatıldığında alt görevlerin "Done" durumuna otomatik geçişi doğrulanacaktır.

3. Timer başlat/durdur davranışı ve süre yuvarlama kuralı (quarter rounding) doğrulanacaktır.

5.11 Zaman İzleme (Time Tracking)

5.11.1 Tanım

Zaman izleme bilgileri hem **görev seviyesinde** hem de **alt görev seviyesinde** tanımlanabilir.

5.11.2 Görev Zaman İzleme (Task Time Tracking)

Görevlerde iki alan bulunur:

1. **Tahmini zaman (Estimated time)**
2. **Harcanan zaman (Spent time)**

Bu değerler çalışma saatlerini temsil eder ve **manuel** olarak ayarlanır.

5.11.3 Alt Görev Zaman İzleme (Subtask Time Tracking)

Alt görevlerde ayrıca:

1. **geçen süre (elapsed/spent)**
2. **zaman tahmini (time estimated)**
   alanları bulunur.

Kural: Bu alanlar değiştirildiğinde, görev zaman izleme değerleri otomatik güncellenir ve tüm alt görev değerlerinin toplamı haline gelir.

5.11.4 Durum Değişimi Bazlı Kayıt (Timesheet / Tracking Table)

Kanboard, alt görev durum değişiklikleri arasındaki zamanı ayrı bir tabloda kaydeder:

1. Durum "Yapılacak → Devam ediyor" olduğunda başlangıç zamanı kaydedilir.
2. Durum "Devam ediyor → Tamamlanmış (Done)" olduğunda bitiş zamanı kaydedilir; alt görevin ve görevin geçen süresi güncellenir.
3. Kayıtların dökümü (timesheet) görev görünüm sayfasında görüntülenir.

5.11.5 Zamanlayıcı (Timer) Kuralları

1. Zamanlayıcı alt görev durumundan bağımsızdır.
2. Zamanlayıcı her başlatıldığında time tracking tablosunda yeni bir kayıt oluşur.
3. Zamanlayıcı durdurulduğunda bitiş tarihi time tracking tablosuna yazılır.
4. Not: "En yakın çeyreğe yuvarlama" ifadesi, Türkçe dokümana göre yalnızca **Kanboard < 1.0.32** için geçerli olarak belirtilmiştir.

5.11.6 API Traceability

Zaman izleme fonksiyonları için API tarafında ilgili procedure grupları bulunmaktadır:

1. **Subtask Time Tracking API Procedures**

5.11.7 Validation Items (Doğrulama Maddeleri)

1. Alt görev zaman verileri değiştiğinde task-level (estimated/spent) toplamlarının doğru güncellendiği doğrulanacaktır.
2. Status-change tabanlı başlangıç/bitiş kayıtlarının ve timesheet görünümünün doğruluğu doğrulanacaktır.
3. Timer bağımsızlığı (status bağımsız start/stop) doğrulanacaktır.

5.12 İç Görev Bağlantıları (Internal Task Links)

5.12.1 Tanım ve Amaç

Görevler, önceden tanımlanmış ilişkiler (task link types) kullanılarak birbirine bağlanabilir. Görevleri **projeler arasında** bağlamak da mümkündür.

5.12.2 Varsayılan İlişki Tipleri (Default Relationships)

Dokümantasyonda belirtilen varsayılan ilişki tipleri:

1. **alakalı**
2. **bloklar | tarafından engellendi**
3. **çoğaltır | tarafından çoğaltılan**
4. **bir çocuğun | bir ebeveyni**
5. **dönüm noktası hedefliyor | bir dönüm noktası**
6. **düzeltmeler | tarafından düzeltildi**

5.12.3 Konfigürasyon Notu

İlişki etiketleri (labels) **uygulama ayarlarından** değiştirilebilir.

5.12.4 API Traceability

1. **Link API Procedures:** İlişki tiplerinin listelenmesi (getAllLinks)
2. **Internal Task Link API Procedures:** Görevler arası link oluşturma/güncelleme gibi işlemler (createTaskLink, updateTaskLink vb.)

5.12.5 Validation Items (Doğrulama Maddeleri)

1. Projeler arası görev linkleme desteği (cross-project linking) UI ve API üzerinden doğrulanacaktır.
2. Uygulama ayarlarında link label değişikliğinin UI ve API çıktısına etkisi doğrulanacaktır.

5.13 Etiketler (Tags)

5.13.1 Tanım ve Kapsam

Kanboard ile bir veya birden fazla etiket (tag) görevle ilişkilendirilebilir. Etiketler:

1. **Global** (tüm projeler için) veya
2. **Project-level** (yalnız belirli bir proje için)
   tanımlanabilir.

5.13.2 UI – Etiket Ekleme / Oluşturma

1. Görev formu üzerinden istenen etiket(ler) girilerek ilişkilendirme yapılabilir.
2. Otomatik tamamlama (autocomplete), kullanılabilir etiketleri önerir.
3. Etiketler doğrudan görev formundan oluşturulabilir; varsayılan davranış olarak bu şekilde oluşturulan etiketler **geçerli proje ile ilişkilendirilir**.

5.13.3 Yönetim Noktaları

1. Tüm etiketler **Project settings** altında yönetilebilir.
2. Etiketleri tüm projeler için **global** tanımlamak için **Application settings** kullanılır.

5.13.4 Arama ve Filtreleme

1. Etiket bazlı arama için “tag” özelliği kullanılır.

5.13.5 API Traceability

1. **Tags API Procedures** (tag CRUD ve task–tag ilişkilendirme)

5.13.6 Validation Items (Doğrulama Maddeleri)

1. Task formundan oluşturulan etiketin default olarak “project tag” davranışı doğrulanacaktır.
2. Global tags ve project tags ayrımının UI/DB/API düzeyinde tutarlılığı doğrulanacaktır.

5.14 Arama ve Filtreleme (Advanced Search Syntax & Custom Filters)

5.14.1 Gelişmiş Arama Sözdizimi (Advanced Search Syntax)

Kanboard, gelişmiş arama için basit bir sorgu dili kullanır. Görevler, yorumlar, alt görevler, bağlantılar ve etkinlik akışında arama yapılabilir.

5.14.1.1 Örnek Sorgu

Örnek sorgu yapısı: assignee:me due:tomorrow <free text>

5.14.1.2 Temel Arama Yetkinlikleri (Dokümantasyonda listelenen başlıklar)

Gelişmiş arama dokümanında aşağıdaki başlıklar bulunur (özellik adları “attribute” olarak geçer):

1. Task ID / title arama (örn. #123, 123, serbest metin)

2. status (open/closed)
3. assignee (me/nobody/username/full name)
4. creator (me/user id/full name)
5. subtask:assignee
6. color
7. due (due date)
8. modified/created/start date aramaları (değiştirilme/oluşturma/başlangıç tarihi)
9. description
10. external reference
11. category
12. project
13. column
14. swimlane
15. task link / comments / tags aramaları

Not: Attribute'ların tüm operator/format detayları (örn. "before/after" gibi) ilgili doküman sayfasındaki örneklere göre test adımlarında detaylandırılacaktır.

5.14.1.3 Etkinlik Akışı (Activity Stream) Arama

Doküman, etkinlik akışı içinde aramayı ayrıca ele alır (başlığa göre, duruma göre, olay yaratıcısına göre, oluşturma tarihine göre, projeye göre).

5.14.2 Özel Filtreler (Custom Filters)

5.14.2.1 Tanım

Özel filtreler, herhangi bir arama sorgusunu kaydetmeye olanak tanır; bu sayede varsayılan filtreler genişletilebilir ve sık kullanılan arama sorguları saklanabilir.

5.14.2.2 Saklama ve Paylaşım Kuralları

1. Özel filtreler **proje tarafından** saklanır ve **oluşturucuyla ilişkilendirilir**.
2. Filtreyi oluşturan kişi **project manager** ise filtreyi diğer proje üyeleriyle paylaşmayı seçebilir.

5.14.2.3 UI – Oluşturma Akışı

Özel filtre oluşturma, "İşlem (Actions) dropdown" veya proje ayarları üzerinden "Özel Filtreler" seçilerek yapılır.
Oluşturulan filtre, panoda varsayılan filtrelerin yanında görünür.

5.14.3 Validation Items (Doğrulama Maddeleri)

1. Arama attribute'larının dokümantasyondaki örneklere uygun parse edildiği doğrulanacaktır.

2. Custom filter görünürlüğü: creator, project manager paylaşımı ve diğer üyelerde görünüm davranışı doğrulanacaktır.

5.15 Görev Geçişleri ve Analizler (Task Transitions & Analytics)

5.15.1 Görev Geçişleri (Task Transitions)

5.15.1.1 Tanım

Bir görevin sütunlar arasındaki her hareketi veritabanına kaydedilir (task transitions).

5.15.1.2 Kayıt Alanları

Görev görünümünden erişilebilen geçiş kayıtlarında aşağıdaki bilgiler bulunur:

1. Eylem tarihi

2. Kaynak sütun

3. Hedef sütun

4. Yürütücü (işlemi yapan kullanıcı)

5. Başlangıç sütununda geçen süre

5.15.2 Görev Bazlı Analiz (Task Analytics)

5.15.2.1 Erişim

Her görev, görev görünümünde sol menüden erişilebilen bir analiz bölümüne sahiptir.

5.15.2.2 Lead Time / Cycle Time Tanımları

1. **Teslim süresi (Lead time):** görev yaratma ile tamamlanma tarihi arasındaki zaman (task kapatıldığında).

2. **Döngü süresi (Cycle time):** başlangıç tarihi ile tamamlanma tarihi arasındaki zaman.

3. Görev kapalı değilse tamamlanma tarihi yerine mevcut saat kullanılır.

4. Başlangıç tarihi belirtilmemişse cycle time hesaplanmaz.

5. Başlangıç tarihinin belirli bir kolona taşıma anında otomatik set edilmesi için otomatik eylem (automatic action) kullanılabileceği not edilmiştir.

5.15.2.3 Kolon Bazlı Süre (Time Spent per Column)

Görev için her bir kolonda geçen toplam süre gösterilir; geçen süre görev kapanana kadar hesaplanır.

5.15.3 Proje Analizleri (Project Analytics)

5.15.3.1 Kapsam

Her projenin bir analiz bölümü vardır; kullanım şekline bağlı olarak çeşitli raporlar görüntülenir.

5.15.3.2 Rapor Türleri (Dokümantasyonda listelenen)

1. Kullanıcı yeniden bölümlendirme (open task count per user)
2. Görev dağıtımı (open task count per column)
3. Kümülatif akış diyagramı (cumulative flow diagram)
4. Burndown chart (karmaşıklık veya story points kullanarak)
5. Her sütuna harcanan ortalama süre (son 1000 görev; transition verisiyle; task kapanana kadar)
6. Ortalama lead/cycle time trendi (son 1000 görev; günlük hesaplanıp kaydedilir)

5.15.3.3 Cumulative Flow – Veri Gereksinimi ve Ayar

1. Her gün her sütun için toplam görev sayısı kaydedilir.
2. Grafiğin görüntülenmesi için en az iki günlük veri gereklidir.
3. Kapatılan görevleri hariç tutmak için “global proje ayarları” üzerinden ilgili seçenek değiştirilir.

5.15.3.4 Cronjob Gereksinimi

Doğru istatistikler için günlük cronjob çalıştırılması gerektiği not edilmiştir.

5.15.4 Validation Items (Doğrulama Maddeleri)

1. Task transitions kayıtlarının: hareket, kullanıcı ve süre alanlarını doğru ürettiği doğrulanacaktır.
2. Task analytics: start date yokken cycle time hesaplanmaması; task açıkken completion yerine “current time” kullanılması doğrulanacaktır.
3. Project analytics: cumulative flow için veri birikimi (≥2 gün), burndown hesap yaklaşımı (complexity/story points), “last 1000 tasks” kapsamı ve cronjob gereksinimi doğrulanacaktır.

5.16 Ekran Görüntüleri Ekleme (Screenshots / Task Attachments)

5.16.1 Tanım

Görüntüler doğrudan Kanboard’a kopyala-yapıştır ile eklenebilir; eklenen görüntüler göreve **ek (attachment)** olarak yüklenir.

5.16.2 UI – Ekleme Yöntemleri

1. Açılır menü (dropdown) üzerinden veya görev görünümü sayfasından doğrudan ekran görüntüsü eklenebilir.

2. Yeni görüntü eklemek için ekran görüntüsü alınıp **CTRL+V** (Windows/Linux) veya **Command+V** (macOS) ile yapıştırılır.

5.16.3 Tarayıcı Uyumluluğu

Bu özellik tüm tarayıcılarda çalışmayabilir; dokümantasyonda Safari ile çalışmadığı (WebKit issue) belirtilmiştir.

5.16.4 API Traceability (Task Attachment)

1. **Task File API Procedures:** Task attachment upload/download/remove vb. işlemler (örn. createTaskFile, getAllTaskFiles, downloadTaskFile, removeTaskFile).

5.16.5 Validation Items (Doğrulama Maddeleri)

1. Paste (CTRL+V / Command+V) ile eklenen görselin task attachments alanında doğru listelendiği doğrulanacaktır.

2. Browser uyumluluğu: Chrome/Firefox/Edge üzerinde çalışma; Safari'de dokümanda belirtilen davranış doğrulanacaktır.

3. API ile upload edilen attachment'ın UI'da görünürlüğü ve UI ile eklenen attachment'ın API'dan listelenmesi doğrulanacaktır.

5.17 RSS / Atom Abonelikleri (RSS / Atom Subscriptions)

5.17.1 Amaç ve Kapsam

Sistem; **proje aktiviteleri** ve **kullanıcı aktiviteleri** için RSS/Atom yayınları (feed) sağlamalıdır. Bu yayınlar, dış RSS okuyucuları ile aktivitelerin takip edilmesini mümkün kılar.

5.17.2 Feed Türleri

1. **Project feed:** Yalnızca ilgili projenin aktivitelerini içerir.

2. **User feed:** Kullanıcının üye olduğu tüm projelerin etkinlik akışını içerir.

5.17.3 Etkinleştirme / Devre Dışı Bırakma (Public Access Bağımlılığı)

RSS abonelikleri, yalnızca **genel erişim (public access)** etkinleştirildiğinde kullanılabilir.

1. **Project RSS:** *Project Settings > Public Access* üzerinden etkinleştirilir/devre dışı bırakılır.

2. **User RSS:** *User Profile > Public Access* üzerinden etkinleştirilir/devre dışı bırakılır.

5.17.4 Güvenlik Gereksinimleri

1. RSS bağlantısı **rastgele bir token** ile korunur; yalnızca URL'yi bilen kişiler özet akışına erişebilir.

5.17.5 Kabul Kriterleri (Acceptance Criteria)

1. **AC-5.17-01:** Public access kapalıyken RSS feed URL'leri erişilebilir olmamalıdır.
2. **AC-5.17-02:** Public access açıldığında proje feed'i yalnızca ilgili proje aktivitelerini göstermelidir.
3. **AC-5.17-03:** Public access açıldığında kullanıcı feed'i kullanıcının üye olduğu tüm projelerin aktivitelerini göstermelidir.
4. **AC-5.17-04:** Feed URL token'ı değiştirilirse önceki URL ile erişim olmamalıdır (token bazlı güvenlik).

5.17.6 Validation Items

1. Token yenileme davranışının (UI üzerinden) nasıl tetiklendiği ve eski token'ın invalidasyonu ortamda doğrulanacaktır.

5.18 Takvim Senkronizasyonu (iCalendar / iCal)

5.18.1 Amaç ve Kapsam

Sistem; projeler ve kullanıcılar için **iCal (iCalendar) feed** üretmelidir. Bu sayede Kanboard görevleri dış takvim uygulamalarında görüntülenebilir. Feed **salt okunur (read-only)** olmalı; harici takvim uygulamasından görev oluşturma desteklenmemelidir.

5.18.2 Veri Aralığı Kısıtı

iCalendar feed'e yalnızca **-2 ay (geçmiş) ile +6 ay (gelecek)** tarih aralığındaki görevler aktarılmalıdır.

5.18.3 Takvim Türleri

1. **Project calendars**
    1. Her projenin kendi takvimi vardır.
    2. Proje abonelik linki benzersizdir ve **Project Settings > Public Access** etkinleştirildiğinde aktif olur.
    3. Yalnızca seçilen projeye ait görevler görünür.
2. **User calendars**
    1. Her kullanıcının kendi takvimi vardır.
    2. Kullanıcı abonelik linki benzersizdir ve **User Profile > Public Access** etkinleştirildiğinde aktif olur.
    3. Kullanıcıya atanan görevleri (tüm projelerden) gösterir.

5.18.4 Dış Takvim Uygulaması Entegrasyon Notları

Dokümantasyonda iCal feed'in takvim istemcilerine eklenmesi için örnek adımlar verilmiştir (örn. Apple Takvim, Microsoft Outlook, Mozilla Thunderbird, Google Takvim).
Not: Google Takvim tarafında harici takvimlerin çok sık yenilenmeyebileceği belirtilir.

5.18.5 Kabul Kriterleri (Acceptance Criteria)

1. **AC-5.18-01:** iCal feed read-only olmalı; takvim yazılımından Kanboard'a task create/update yapılamamalıdır.

2. **AC-5.18-02:** iCal feed yalnız -2 ay / +6 ay aralığını kapsamalıdır.

3. **AC-5.18-03:** Project calendar linki sadece ilgili proje için görevleri göstermelidir.

4. **AC-5.18-04:** User calendar linki kullanıcıya atanan görevleri (tüm projelerden) göstermelidir.

5. **AC-5.18-05:** Public access kapalıyken iCal abonelik linki aktif olmamalıdır.

5.18.6 Validation Items

1. iCal feed'de "tarih alanı"nın hangi task tarihlerini kullandığı (due date / start date vb. görünümler) takvim çıktısı üzerinden doğrulanacaktır.

5.19 E-posta ile Görevler Oluşturma (Create Tasks by Email)

5.19.1 Amaç ve Kapsam

Sistem, bir e-posta gönderilerek görev oluşturulmasını desteklemelidir. Bu özellik **çekirdek sistemde değil**, **plugin + üçüncü parti inbound email servisleri** üzerinden sağlanır.

5.19.2 Desteklenen Inbound Servisler

Dokümantasyonda 3 entegrasyon listelenir:

1. Mailgun

2. SendGrid

3. Postmark

Bu servisler, gelen e-postaları işleyerek Kanboard'a iletir; ayrıca "SMTP sunucusu kurmadan" akışı mümkün kılar.

5.19.3 İş Akışı (Inbound Email Workflow)

1. Belirli bir adrese e-posta gönderilir (örn. something+myproject@inbound...).

2. E-posta üçüncü parti SMTP/Inbound servisine yönlendirilir.

3. Sağlayıcı, Kanboard **webhook (hook)** çağırarak e-postayı JSON veya multipart/form-data formatında gönderir.

4. Kanboard e-postayı ayrıştırır ve doğru projeye göre bir görev oluşturur.

Kural: Yeni görevler otomatik olarak **ilk sütunda** oluşturulur.

5.19.4 E-posta Biçimi (Parsing Rules)

1. E-posta adresinin local-part'ında **plus separator** kullanılmalıdır (örn. kanboard+project123).

2. + sonrasındaki ifade, proje tanımlayıcısı ile eşleşmelidir (örn. project123).

3. E-posta **subject** → task **title** olur.

4. E-posta **body** → task **description** olur (Markdown).

5. Gelen e-postalar text veya HTML olabilir; Kanboard basit HTML'i Markdown'a çevirebilir.

5.19.5 Güvenlik ve Gereksinimler

1. Kanboard webhook'u rastgele bir **token** ile korunur.

2. Gönderen e-posta adresi bir Kanboard kullanıcısı ile eşleşmelidir.

3. Proje benzersiz bir tanımlayıcıya sahip olmalıdır (örn. "PROJEM").

4. İlgili kullanıcının projeye **üye** olması gerekir.

5.19.6 Kabul Kriterleri (Acceptance Criteria)

1. **AC-5.19-01:** Subject → title, body → description (Markdown) dönüşümü doğru olmalıdır.

2. **AC-5.19-02:** Task varsayılan olarak ilk sütunda açılmalıdır.

3. **AC-5.19-03:** Plus separator ve proje identifier eşleşmesi yoksa task oluşturulmamalıdır.

4. **AC-5.19-04:** Gönderen kullanıcı sistemde yoksa veya projeye üye değilse task oluşturulmamalıdır.

5. **AC-5.19-05:** Webhook token hatalıysa istek reddedilmelidir.

5.19.7 Validation Items

1. Plugin kurulum/konfigürasyon adımlarının projedeki hedef ortama (Docker/VM/Reverse proxy) göre farklılaşması beklenir; ilgili entegrasyonun gerçek ortamda çalıştığı evidence ile doğrulanacaktır.

5.20 Çift-Kademeli Kimlik Doğrulama (2FA / TOTP)

5.20.1 Amaç ve Kapsam

Her kullanıcı, hesabı için **Çift-Kademeli Kimlik Doğrulama (two-factor authentication)** özelliğini etkinleştirebilmelidir. Başarılı kullanıcı adı/parola doğrulamasından sonra Kanboard erişimi için **tek kullanımlık 6 karakter kod** istenmelidir.

5.20.2 Teknik Yöntem (TOTP)

Kanboard, RFC 6238'de tanımlanan **TOTP (Time-based One-time Password)** algoritmasını kullanır.

5.20.3 Kurulum Akışı (User-level Setup)

1. Kullanıcı profiline gidilir.

2. Sol menüden "İki faktörlü kimlik doğrulama" seçilir ve etkinleştirilir.

3. Sistem bir **secret key** üretir.

4. Kullanıcı TOTP uygulamasına secret key'i kaydeder (mobilde QR kod ile).

5.20.4 Davranış Kuralları

1. 2FA açıkken her yeni oturum açılışında kod sorulmalıdır.
2. 2FA her enable/disable işleminde **yeni bir secret key** üretilmelidir.
3. TOTP sistemi çevrimdışı çalışabilir; zorunlu olarak telefon gerektirmez (uygun TOTP aracı yeterlidir).

5.20.5 Uyumlu TOTP Uygulamaları (Örnekler)

Dokümantasyon örnek olarak şu uygulamaları verir: Google Authenticator, FreeOTP ve OATH Toolkit (CLI).

5.20.6 Kabul Kriterleri (Acceptance Criteria)

1. **AC-5.20-01:** 2FA etkinleştirildiğinde sonraki login akışında 6 karakter TOTP kodu zorunlu olmalıdır.
2. **AC-5.20-02:** Yanlış TOTP kodu ile giriş tamamlanmamalıdır.
3. **AC-5.20-03:** 2FA disable → enable yapıldığında yeni secret key üretildiği doğrulanmalıdır.
4. **AC-5.20-04:** Kullanıcı 2FA'yı kapatırsa login akışı tekrar yalnız username/password ile tamamlanabilmelidir.

5.20.7 Validation Items

1. 2FA açık kullanıcılar için API authentication yaklaşımının (password vs personal access token) proje politikasına göre netleştirilmesi ve test senaryolarına işlenmesi gerekir (özellikle RBAC API testlerinde).

5.21 Bildirimler (Notifications) ve Kullanıcı Bahsetmeleri (User Mentions)

5.21.1 Amaç ve Kapsam

Sistem, proje ve görev aktiviteleri hakkında kullanıcıların haberdar edilmesi için bildirim (notification) mekanizması sağlamalıdır. Bildirimler; **Email** ve **Web notifications** kanalları üzerinden iletilebilir. Harici eklentiler ile mesajlaşma sistemlerine bildirim gönderimi desteklenebilir.

5.21.2 Bildirim Kanalları (Notification Channels)

Sistem, aşağıdaki temel kanalları desteklemelidir:

1. **Email**
2. **Web** (okunmamış bildirimlerin listesi)

Harici eklenti ile aşağıdaki mesajlaşma sistemlerine bildirim gönderimi sağlanabilir: Slack, Hipchat, Jabber.

5.21.3 Konfigürasyon (User-level Configuration)

1. Bildirimler **varsayılan olarak kapalıdır**; her kullanıcı kendi profilinden etkinleştirmelidir: *User Profile > Notifications*.

2. Email bildirimleri için kullanıcı profilinde **geçerli bir e-posta adresi** bulunmalıdır ve uygulama e-posta gönderecek şekilde **konfigüre edilmiş** olmalıdır.

3. Kullanıcı, tercih ettiği bildirim yöntemini seçebilmelidir: **Email** veya **Web**.

4. Kullanıcı, üyesi olduğu projeler için bildirim kapsamını aşağıdaki seçeneklerle belirleyebilmelidir:

    1. **All tasks**

    2. **Only tasks assigned to you**

    3. **Only tasks created by you**

    4. **Only tasks created by you and assigned to you**

5. Kullanıcı, bildirim alacağı projeleri ayrıca seçebilmelidir; varsayılan olarak üye olunan projeler seçili kabul edilir.

5.21.4 Web Bildirimleri (Web Notifications)

1. Web bildirimleri **dashboard'dan** veya üstteki **bildirim ikonundan** erişilebilir olmalıdır.

2. Bildirimler liste şeklinde gösterilmeli; kullanıcı tek tek **okundu (mark as read)** işaretleyebilmeli veya tümünü **toplu** okundu işaretleyebilmelidir.

3. Web bildirimleri, email almadan bildirim takibini mümkün kılmalıdır.

5.21.5 Kullanıcı Bahsetmeleri (User Mentions)

1. Sistem, görev veya yorum oluşturma sırasında @username formatıyla kullanıcı bahsetmesini desteklemelidir. Sistem, yazım esnasında kullanıcı listesi önerebilmelidir (auto-suggest).

2. Bahsetme özelliği dokümantasyona göre yalnızca şu alanlarda etkin olmalıdır:

    1. **Task description**

    2. **Comment text area**

3. Bahsetme yalnızca **task ve comment creation** sırasında çalışmalıdır.

4. Bildirim alabilmesi için bahsedilen kullanıcının ilgili projeye **üye** olması gerekir.

5. Bahsedilen kullanıcı, kendi bildirim ayarlarına uygun kanaldan bildirim almalıdır (Email/Web/Plugin).

6. @username bahsetmesi, public user profile bağlantısına yönlenmelidir.

5.21.6 Email Bildirimlerinde Link Görünürlüğü (Application URL)

1. Email bildirimlerinde görev linkinin görünebilmesi için **Application URL** ayarının tanımlı olması gerekir.

2. Bu ayar *Settings > Application settings* altında yönetilir ve email footer'da görev linki üretiminde kullanılır.

5.21.7 Overdue Task Notifications (CLI üzerinden – Opsiyonel Operasyonel Akış)

Sistem, gecikmiş görevler için bildirim gönderimini komut satırı üzerinden tetikleyebilir:

1. ./cli notification:overdue-tasks komutu, bildirimleri etkin kullanıcıların e-posta adreslerine gecikmiş görev bildirimleri gönderebilir.

2. Ek parametreler ile "tek kullanıcıya toplu", "proje yöneticilerine toplu", "belirli proje ile filtreleme" gibi davranışlar uygulanabilir.

Not: Bu madde, kurulumun CLI kullanacak şekilde planlanması halinde "Operational Notifications" kapsamına dahildir.

5.21.8 Kabul Kriterleri (Acceptance Criteria)

1. **AC-5.21-01:** Yeni kullanıcıda notifications default **disabled** olmalıdır; kullanıcı profilden enable ettiğinde bildirim üretimi başlamalıdır.

2. **AC-5.21-02:** Email bildirimleri için profil email alanı boşsa, email bildirim akışı çalışmamalı ve sistem davranışı dokümana uygun olmalıdır (konfigürasyon gereksinimi).

3. **AC-5.21-03:** Web notifications listesi dashboard/ikon üzerinden erişilebilir olmalı; tekil ve toplu okundu işaretleme çalışmalıdır.

4. **AC-5.21-04:** @username mention yalnızca task description ve comment alanlarında, oluşturma aşamasında çalışmalı; bahsedilen kişi proje üyesi ise bildirim almalıdır.

5. **AC-5.21-05:** Application URL tanımlı değilse email bildirimlerinde görev linki üretilmemeli; tanımlandığında görev linki görünmelidir.

6. **AC-5.21-06:** notification:overdue-tasks komutu çalıştırıldığında, notifications enabled kullanıcılarına overdue task bildirimleri e-posta ile gönderilmelidir.

5.21.9 Validation Items (Doğrulama Maddeleri)

1. Harici plugin kanalları (Slack/Hipchat/Jabber) kurulumu kapsamda ise, kanala göre delivery ve message format doğrulanacaktır.

2. Email gönderim altyapısı (SMTP / provider / container kısıtları) ortam mimarisine göre doğrulanacaktır. (Örn. Docker ortamında mail/sendmail transport kısıtı gibi durumlar.)

3. User API ile başarısız auth denemeleri sonrası “account unlock via login form” gereksinimi (brute force koruması ile ilişkili) test edilecektir.

5.22 Master Traceability Matrix (BA → UI → API → Test Type)

5.22.1 Amaç

Bu matris; BA kapsamındaki her fonksiyonun ilgili UI alanı, API procedure group’u ve test tipi ile izlenebilirliğini sağlar.

5.22.2 Matris (Örnek / Başlangıç)

BA Bölümü	UI/Feature	API Procedure Group	Test Type
5.6 Automatic Actions	Project settings > Automatic actions	Action Procedures	UI + API + Negative
5.7 API Layer	JSON-RPC endpoint/auth	(Tüm gruplar)	API + Security
5.8 Authorization	Roles/Permissions screens	Project Permission / User / Group	RBAC + Negative
5.10 Subtasks	Task view > Subtasks	Subtask / Subtask Time Tracking	UI + API + Data Integrity
5.12 Task Links	Task view > Links	Link / Internal Task Link	UI + API
5.13 Tags	Task form > Tags	Tags	UI + API
5.14 Search & Filters	Search bar / Filters	(UI-driven)	UI + Query Validation
5.16 Attachments	Task view > Files/Screenshots	Task File	UI + API
5.21 Notifications	Profile > Notifications / Web icon	(UI-driven)	UI + Email/Web delivery

Matris, kapsam “tamamlama” aşamasında tüm BA başlıklarını kapsayacak şekilde genişletilecektir.

5.23 Test Coverage Matrix (Kapsam Doğrulama Matrisi)

5.23.1 Amaç

Bu matris; her fonksiyonel alan için planlanan doğrulama tiplerini (UI/API/RBAC/Negative/Data Integrity/Compatibility/Operational) standardize eder ve "tam kapsam" hedefi için test boşluğu bırakılmamasını sağlar.

5.23.2 Test Tipleri (Tanımlar)

1. **UI Functional:** Web arayüzündeki akışların ve ekran davranışlarının doğrulanması
2. **API Functional:** JSON-RPC API prosedürlerinin sözleşmeye uygunluğu (params/result)
3. **RBAC / Authorization:** Role & project permissions bazlı erişim/doğrulama (allowed/denied)
4. **Negative:** Hatalı parametre, invalid ID, yetkisiz erişim, invalid state vb. durumlar
5. **Data Integrity:** UI ve API üzerinden yapılan işlemlerde verinin tutarlılığı (UI–API consistency dahil)
6. **Compatibility:** Tarayıcı/istemci uyumluluğu (özellikle clipboard/paste, dosya ekleri)
7. **Operational:** CLI/Cronjob gibi operasyonel işlerin doğru çalışması (analytics, overdue notifications vb.)

5.23.3 Kapsam Matrisi

Feature Area	UI	API	RBAC	Negative	Data Integrity	Compatibility	Operational
Projects / Project Settings	✓	✓	✓	✓	✓	—	—
Board / Columns / Limits	✓	✓	✓	✓	✓	—	—
Swimlanes	✓	✓	✓	✓	✓	—	—
Tasks (CRUD, move, close/open)	✓	✓	✓	✓	✓	—	—
Subtasks	✓	✓	✓	✓	✓	—	—
Time Tracking / Timers	✓	✓	✓	✓	✓	—	✓
Tags	✓	✓	✓	✓	✓	—	—
Internal Task Links	✓	✓	✓	✓	✓	—	—
Search / Custom Filters	✓	—	✓	✓	—	—	—
Automatic Actions	✓	✓	✓	✓	✓	—	✓

Feature Area	UI	API	RBAC	Negative	Data Integrity	Compatibility	Operational
Notifications (Email/Web)	✓	—	✓	✓	—	—	✓
Mentions (@username)	✓	—	✓	✓	—	—	—
RSS/Atom Public Feeds	✓	—	✓	✓	—	—	—
iCal Public Feeds	✓	—	✓	✓	—	—	—
Attachments / Screenshots (paste)	✓	✓	✓	✓	✓	✓	—

Not: Notifications/Mentions için dokümantasyon UI davranışlarını net tanımlar; ayrıca "mentioned users must be members of the project" ve "mentions only during task/comment creation" kuralları doğrulama kapsamındadır.

5.23.4 Bu Proje İçin Test Edilebilirlik Notu

SMTP/inbound email/cron altyapısı kurulmadığı için 'Operational' ve 'Create Tasks by Email' gibi dış bağımlılığı olan alanlar execution ile doğrulanamayabilir. Bu alanlar matriste 'Not Executed (Environment Constraint)' olarak işaretlenir ve doğrulama yöntemi dokümantasyon incelemesi olarak kayıt altına alınır.

5.24 Test Deliverables (Çıktılar)

5.24.1 Zorunlu Deliverable'lar

1. **BA Traceability Matrix** (BA → UI → API → Test Type) (Bkz. 5.22)
2. **Test Case Repository**
    1. UI test case seti (manuel + otomasyon kapsamı ayrımı)
    2. API test case seti (JSON-RPC prosedür grupları bazında)
    3. RBAC matrix test case seti (role × action)
3. **Test Evidence Pack**
    1. UI ekran görüntüleri / video / log çıktıları
    2. API request/response örnekleri
4. **Defect Reports**
    1. Repro steps + expected/actual + environment + evidence
5. **Test Summary Report**

    1. Execution summary, pass/fail oranları, blocker listesi, riskler, öneriler

---

5.25 Test Environment & Test Data (Ortam ve Veri)

5.25.1 Test Ortamı Gereksinimleri

1. Kanboard instance (test ortamı)
2. En az 1 adet "team project" ve 1 adet "private project" (permission davranışı için)
3. API erişimi için:
    1. Application API token (jsonrpc user)
    2. User API credential / personal access token (RBAC testleri için)

5.25.2 Minimum Test Kullanıcı Seti

1. **Administrator**
2. **Manager**
3. **User**
4. Proje seviyesinde:
    1. **Project Manager**
    2. **Project Member**
    3. **Project Viewer**
5. En az 1 adet **Custom Project Role** (restriction setleri test edilecek)

5.25.3 Minimum Test Veri Seti

1. En az 3 sütunlu board (örn. Backlog / In Progress / Done)
2. En az 2 swimlane (default + ek swimlane)
3. En az 10 task (çeşitli durum/kolon/swimlane kombinasyonları)
4. En az 5 subtask (timer ve status-change tracking için)
5. En az 5 tag (project + global ayrımı için)
6. En az 3 internal task link (bloklar/engellendi, parent/child vb.)
7. En az 2 otomatik aksiyon (kolona taşınma → close/assign/start date)

5.25.4 Bu Proje İçin Ortam Kısıtları

Bu çalışma bireysel bitirme projesi kapsamında tek kişi tarafından yürütülmektedir. Bu nedenle test ortamı, elde kurulan lokal Kanboard instance ile sınırlıdır.

Aşağıdaki bileşenler bu çalışma kapsamında kurulmayacaktır / mevcut değildir:

- Outbound email / SMTP altyapısı (email bildirim testleri UI seviyesinde sınırlı kalacaktır).

- Inbound email servisleri (Mailgun/SendGrid/Postmark) ve ilgili plugin entegrasyonları (Create Tasks by Email) - execution N/A, dokümantasyon incelemesi ile doğrulanacaktır.

- Scheduled cron servisleri ve üretim benzeri job altyapısı (cronjob/CLI ile çalışan operational senaryolar, erişim varsa doğrulanır; yoksa N/A olarak raporlanır).

Bu kısıtlar, kapsamı değiştirmez; yalnızca bazı senaryolarda doğrulama yöntemini (execution yerine dokümantasyon bazlı doğrulama) etkiler.

5.26 Execution Approach (Uygulama Planı)

5.26.1 Test Fazları

1. **Smoke (Giriş Kontrolü)**
    1. UI login, temel project/task oluşturma
    2. API connectivity (jsonrpc endpoint + getAllProjects)
2. **Functional UI**
    1. Board/Task/Subtask/Time/Tags/Links/Actions/Notifications
3. **API Functional**
    1. Procedure group bazında CRUD + “Result on failure” davranışları
4. **RBAC / Authorization**
    1. User API ile 403/allowed doğrulaması
5. **Data Integrity (UI–API Consistency)**
    1. UI’da oluştur → API’de doğrula; API’de oluştur → UI’da doğrula
6. **Regression**
    1. Kritik akışların tekrar çalıştırılması (release adayı)

5.26.2 Kısıtlı Ortamda Doğrulama Stratejisi

Bu projede SMTP/inbound email/cron altyapısı kurulmadığı için ilgili feature'lar aşağıdaki yöntemlerle doğrulanacaktır:

- Dokümantasyon bazlı doğrulama: Özelliklerin beklenen davranışları resmi dokümantasyondan referanslanır ve BA/Test Scope içinde kayıt altına alınır.

- UI konfigürasyon kontrolü: Uygulama içinde ilgili ayar ekranlarının varlığı ve erişim yetkileri doğrulanır (execution gerektiren kısımlar N/A).

- N/A raporlama: Teknik olarak tetiklenemeyen senaryolar 'Not Executed (Environment Constraint)' olarak işaretlenir ve gerekçe yazılır.

5.27 Operational Requirements (Cronjob / CLI)

5.27.1 Daily Cronjob Requirement

Kanboard'un doğru çalışması için günlük arka plan işinin düzenli çalıştırılması gereklidir. Bu iş; özellikle **reports & analytics**, **overdue task notifications** ve "Daily background job for tasks" event'ine bağlı **automatic actions** için gereklidir.

Dokümantasyon örneği (Linux/Unix):
./cli cronjob komutunun günlük olarak cron üzerinden çalıştırılması.

5.27.2 Overdue Task Notifications (CLI)

Overdue task bildirimleri, CLI üzerinden notification:overdue-tasks komutu ile gönderilebilir. Komut; notifications enabled kullanıcılarına email gönderir ve opsiyonel olarak --group, --manager, --project gibi parametreleri destekler.

5.27.3 Bu Proje İçin Uygulanabilirlik Notu

Bu proje kapsamında cron servisleri ve email gönderimi (SMTP) kurulmadığı için 5.27 altındaki operasyonel senaryoların bir kısmı execution ile doğrulanamayabilir. Erişim yoksa sonuçlar 'Not Executed (Environment Constraint)' olarak raporlanacaktır.

5.28 Entry / Exit Criteria (Başlama ve Bitirme Kriterleri)

5.28.1 Entry Criteria

1. Lokal Kanboard instance kurulu ve erişilebilir
2. Minimum test kullanıcıları ve projeler hazırlanmış (team project + personal project)
3. API token/credentials doğrulanmış
4. Opsiyonel: Cronjob/CLI çalıştırma yetkisi varsa operational testler yürütülecek; yoksa N/A olarak raporlanacaktır

5.28.2 Exit Criteria

1. Kritik akışlarda (Projects/Board/Tasks/Subtasks/Time/Actions/API/RBAC) blocker defect kalmamış
2. RBAC matrisindeki "denied" senaryolarında 403/engelleme davranışı tutarlı
3. UI–API consistency kontrollerinde kritik veri tutarsızlığı bulunmuyor

4. Operational (cronjob + overdue notifications) doğrulandı (eğer ortam izin veriyorsa); aksi halde 'Not Executed (Environment Constraint)' olarak raporlandı

5. Test Summary Report yayımlandı (5.24)

5.29 Open Items & Validation Items (Açık Maddeler)

1. Role çakışması / doküman farklılığı görülen alanlar: UI + User API ile doğrulanıp matrise netleştirilecek (ilgili BA maddelerinde “Validation Item” işaretli).

2. JSON-RPC “error object” alan şeması: gerçek sistem çıktısı ile Appendix’e eklenecek (örnek hata senaryoları üzerinden).

3. Notifications/Mentions: Mention’ın yalnız task description + comment text area’da ve yalnız creation sırasında çalıştığı; mention edilen kişinin projeye üye olması gerektiği doğrulanacak.

6. Non-Functional Requirements (NFR) ve Operasyonel Varsayımlar

Bu bölüm, ürünün davranışını etkileyen ancak fonksiyonel akış olmayan gereksinimleri listeler. Bu bitirme projesinde tek kişi çalışıldığı ve test ortamı/altyapı kısıtlı olduğu için bazı maddeler yalnız dokümantasyon incelemesi ve yerel doğrulama ile ele alınacak; altyapı gerektirenler 'Not Executed (Environment Constraint)' olarak raporlanacaktır.

6.1 Security

- HTTPS/TLS: Uygulama trafiği mümkünse TLS üzerinden çalıştırılmalıdır. (Local ortamda self-signed sertifika kabul edilebilir.)

- Brute force protection: Login form ve User API başarısız deneme davranışı doğrulanacaktır (CAPTCHA/lock/unlock akışı).

- API authentication: Basic Auth veya opsiyonel custom header. RBAC doğrulaması User API ile yapılır; Application API RBAC için kullanılmaz.

6.2 Background Workers / Cronjobs

- Günlük cronjob (./cli cronjob) rapor/istatistik güncellemeleri ve bazı otomatik aksiyonlar için önerilir. Bu proje kapsamında cron altyapısı kurulmadıysa 'Not Executed (Environment Constraint)'.

- Cronjob yoksa: UI davranışı ve temel işlevler test edilir; analytics güncellemeleri ve overdue notification gibi çıktılar koşullu olarak raporlanır.

6.3 Email / SMTP Configuration

- SMTP/Outbound email: Bu bitirme projesinde SMTP kurulumu yoksa email bildirimleri ve overdue email gönderimi 'Not Executed (Environment Constraint)'.

- Application URL: Email footer'daki task linkinin doğru üretimi için Application URL ayarı gerekir. SMTP yoksa bu madde konfigürasyon kontrolü olarak doğrulanır (UI ayarı mevcut mu?).

6.4 Logging / Audit / Observability

- Hata senaryolarında server log'larının (web server/PHP) incelenebilmesi beklenir. Bu proje kapsamında log erişimi sınırlıysa, API response ve UI evidence temel kanıt olarak kullanılır.

6.5 Browser Compatibility

- Screenshots/paste ve dosya upload gibi alanlarda tarayıcı farklılıkları olabilir. En az 1 modern Chromium tabanlı tarayıcı ile doğrulama yapılır; farklı tarayıcı erişimi yoksa 'Limited coverage' notu düşülür.

7. Risk Analizi ve Test Öncelikleri

Bu bölüm, modülleri etki ve olasılığa göre sınıflandırır ve tek kişi/yerel ortam kısıtlarında test önceliğini belirler.

Modül / Alan	Risk Tanımı	Impact	Likelihood	Priority
Authorization & Roles (RBAC)	Yetkisiz erişim, rol kısıtlarının atlanması, veri sızıntısı	High	Medium	P0
Tasks/Board Core	Task create/move/close hataları iş akışını durdurur	High	High	P0
API Layer (JSON-RPC)	Yanlış auth/param handling entegrasyonları bozar	High	Medium	P0
Attachments/Screenshots	Dosya güvenliği/bozuk upload, veri kaybı	Medium	Medium	P1
Automatic Actions	Yanlış otomasyonla hatalı atama/kapama	Medium	Medium	P1
Search/Filters	Yanlış filtre sonuçları raporlama hatası doğurur	Medium	Low	P2
RSS/iCal Public Access	Token sızıntısı ile bilgi ifşası riski	Medium	Low	P2
Email-to-Task / Email notifications	Yanlış proje eşleşmesi/kimlik doğrulama (ortam varsa)	Medium	Low	P2 (Env)
Cron/Analytics	İstatistik	Low	Medium	P3 (Env)

	güncelleme eksikliği (cron yoksa)			

7.1 Test Öncelik Sırası (Tek Kişi / Kısıtlı Ortam)

P0: RBAC + Tasks/Board core + API auth/contract. P1: Attachments + Automatic actions. P2: Search/Filters + Public feeds. P3: Cron/analytics ve email tabanlı akışlar (ortam varsa).

5.30 Ek – API Test Şablonları (JSON-RPC)

5.30.1 API Endpoint ve Authentication

API Endpoint (varsayılan):

1. http://<KANBOARD_BASE_URL>/jsonrpc.php

Application API (jsonrpc kullanıcısı + token):

1. Uygulama ayarlarında görülen jsonrpc kullanıcısı ve token ile Basic Auth yapılır.
2. Bu modda **permissions kontrolü yapılmaz** (RBAC testleri için uygun değildir).

User API (normal kullanıcı + parola):

1. Basic Auth ile username:password kullanılır.
2. Bu modda permissions/RBAC davranışı gözlemlenir.

Opsiyonel – Custom Header ile auth:

1. config.php içinde API_AUTHENTICATION_HEADER tanımlanarak Basic Auth yerine header kullanılabilir.

5.30.2 Standart JSON-RPC Request Zarfı

Zorunlu alanlar:

1. jsonrpc: "2.0"
2. method: prosedür adı (örn. getAllProjects)
3. id: client request id
4. params: **bazı metotlarda object**, bazı metotlarda **array** formatında kullanılır (dokümandaki örnek format esas alınır).

5.30.3 Smoke Pack (Bağlantı / Kimlik Doğrulama)

A) Application API ile getAllProjects

```
curl \
  -u "jsonrpc:<JSONRPC_TOKEN>" \
```

```
  -d '{"jsonrpc":"2.0","method":"getAllProjects","id":1}' \
  http://<KANBOARD_BASE_URL>/jsonrpc.php
```

B) User API ile getMyProjects

```
curl \
  -u "<USERNAME>:<PASSWORD>" \
  -d '{"jsonrpc":"2.0","method":"getMyProjects","id":1}' \
  http://<KANBOARD_BASE_URL>/jsonrpc.php
```

C) User API ile getMe

```
curl \
  -u "<USERNAME>:<PASSWORD>" \
  -d '{"jsonrpc":"2.0","method":"getMe","id":1}' \
  http://<KANBOARD_BASE_URL>/jsonrpc.php
```

D) Custom Header ile getAllProjects (opsiyonel)

```
curl \
  -H 'X-API-Auth: <BASE64(jsonrpc:token)>' \
  -d '{"jsonrpc":"2.0","method":"getAllProjects","id":1}' \
  http://<KANBOARD_BASE_URL>/jsonrpc.php
```

5.30.4 Project Lifecycle Şablonları

createProject

```
{
  "jsonrpc": "2.0",
  "method": "createProject",
  "id": 1001,
  "params": { "name": "BA-TEST Project" }
}
```

updateProject

```
{
  "jsonrpc": "2.0",
  "method": "updateProject",
  "id": 1002,
  "params": { "project_id": 1, "name": "BA-TEST Project Updated" }
}
```

enableProjectPublicAccess / disableProjectPublicAccess

```
{ "jsonrpc":"2.0","method":"enableProjectPublicAccess","id":1003,"params":["1"] }
```

```
{ "jsonrpc":"2.0","method":"disableProjectPublicAccess","id":1004,"params":["1"] }
```

removeProject

```
{
  "jsonrpc": "2.0",
  "method": "removeProject",
  "id": 1005,
  "params": { "project_id": 1 }
}
```

5.30.5 Board / Column / Swimlane Şablonları

getBoard

```
{ "jsonrpc":"2.0","method":"getBoard","id":1101,"params":[1] }
```

getColumns

```
{ "jsonrpc":"2.0","method":"getColumns","id":1102,"params":[1] }
```

addColumn

```
{ "jsonrpc":"2.0","method":"addColumn","id":1103,"params":[1,"Ready"] }
```

updateColumn (task_limit örnekli)

```
{ "jsonrpc":"2.0","method":"updateColumn","id":1104,"params":[2,"In Progress",5] }
```

getActiveSwimlanes

```
{ "jsonrpc":"2.0","method":"getActiveSwimlanes","id":1105,"params":[1] }
```

addSwimlane

```
{ "jsonrpc":"2.0","method":"addSwimlane","id":1106,"params":[1,"Swimlane A"] }
```

disableSwimlane

```
{ "jsonrpc":"2.0","method":"disableSwimlane","id":1107,"params":[1,2] }
```

removeSwimlane

```
{ "jsonrpc":"2.0","method":"removeSwimlane","id":1108,"params":[1,2] }
```

5.30.6 Project Permission (RBAC) Şablonları

getAssignableUsers

```
{ "jsonrpc":"2.0","method":"getAssignableUsers","id":1201,"params":["1"] }
```

addProjectUser (role optional)

```
{ "jsonrpc":"2.0","method":"addProjectUser","id":1202,"params":["1","10","project-viewer"] }
```

changeProjectUserRole

{ "jsonrpc":"2.0","method":"changeProjectUserRole","id":1203,"params":["1","10","project-member"] }

removeProjectUser

{ "jsonrpc":"2.0","method":"removeProjectUser","id":1204,"params":[1,10] }

5.30.7 Task Lifecycle Şablonları

createTask

{
 "jsonrpc":"2.0",
 "method":"createTask",
 "id":1301,
 "params":{
  "title":"BA-TEST Task",
  "project_id":1,
  "column_id":2,
  "color_id":"green",
  "description":"Markdown description",
  "date_due":"2024-07-01 14:25"
 }
}

getTask

{ "jsonrpc":"2.0","method":"getTask","id":1302,"params":{"task_id":1} }

updateTask

{
 "jsonrpc":"2.0",
 "method":"updateTask",
 "id":1303,
 "params":{"id":1,"color_id":"blue","date_due":"2024-07-01 14:25"}
}

moveTaskPosition (column/position/swimlane)

{
 "jsonrpc":"2.0",
 "method":"moveTaskPosition",
 "id":1304,
 "params":{"project_id":1,"task_id":1,"column_id":2,"position":1,"swimlane_id":1}
}

closeTask / openTask

{ "jsonrpc":"2.0","method":"closeTask","id":1305,"params":{"task_id":1} }

{ "jsonrpc":"2.0","method":"openTask","id":1306,"params":{"task_id":1} }

removeTask

{ "jsonrpc":"2.0","method":"removeTask","id":1307,"params":{"task_id":1} }

searchTasks

{
"jsonrpc":"2.0","method":"searchTasks","id":1308,"params":{"project_id":1,"query":"assignee:nobody"} }

moveTaskToProject (array params örnek)

{ "jsonrpc":"2.0","method":"moveTaskToProject","id":1309,"params":[4,5] }

duplicateTaskToProject (array params örnek)

{ "jsonrpc":"2.0","method":"duplicateTaskToProject","id":1310,"params":[5,7] }

5.30.8 Tags Şablonları

setTaskTags

{ "jsonrpc":"2.0","method":"setTaskTags","id":1401,"params":[39,17,["tag1","tag2"]] }

getTaskTags

{ "jsonrpc":"2.0","method":"getTaskTags","id":1402,"params":[17] }

5.30.9 Internal Task Links Şablonları

getAllLinks (link_id sözlüğü)

{ "jsonrpc":"2.0","method":"getAllLinks","id":1501 }

createTaskLink

{ "jsonrpc":"2.0","method":"createTaskLink","id":1502,"params":[2,3,1] }

getAllTaskLinks (task'a bağlı tüm linkler)

{ "jsonrpc":"2.0","method":"getAllTaskLinks","id":1503,"params":[2] }

removeTaskLink

{ "jsonrpc":"2.0","method":"removeTaskLink","id":1504,"params":[1] }

5.30.10 Files (Task File / Project File) Şablonları

createTaskFile (base64 content)

{
 "jsonrpc":"2.0",
 "method":"createTaskFile",
 "id":1601,
 "params":[1,1,"My file","cGxhaW4gdGV4dCBmaWxl"]
}

downloadTaskFile

{ "jsonrpc":"2.0","method":"downloadTaskFile","id":1602,"params":["1"] }

removeTaskFile

{ "jsonrpc":"2.0","method":"removeTaskFile","id":1603,"params":["1"] }

createProjectFile

{
 "jsonrpc":"2.0",
 "method":"createProjectFile",
 "id":1604,
 "params":[1,"My file","cGxhaW4gdGV4dCBmaWxl"]
}

5.30.11 Automatic Actions Şablonları

getAvailableActions

{ "jsonrpc":"2.0","method":"getAvailableActions","id":1701 }

getAvailableActionEvents

{ "jsonrpc":"2.0","method":"getAvailableActionEvents","id":1702 }

getCompatibleActionEvents

{
"jsonrpc":"2.0","method":"getCompatibleActionEvents","id":1703,"params":["\\Kanboard\\Action\\TaskClose"] }

getActions (project bazlı mevcut action'lar)

{ "jsonrpc":"2.0","method":"getActions","id":1704,"params":["1"] }

createAction

{
 "jsonrpc":"2.0",
 "method":"createAction",
 "id":1705,
 "params":{

"project_id":"2",
"event_name":"task.move.column",
"action_name":"\\Kanboard\\Action\\TaskAssignSpecificUser",
"params":{"column_id":"3","user_id":"2"}
}
}

removeAction

{ "jsonrpc":"2.0","method":"removeAction","id":1706,"params":[1] }

5.30.12 RBAC İçin Kritik Kural (Test Notu)

1. **RBAC/permissions doğrulaması** yapılacak tüm API testleri **User API** (normal kullanıcı) ile yürütülmelidir.
2. **Application API** (jsonrpc token) "permissions checked değildir"; RBAC testi için kullanılmaz.

5.31 Ek – API Negative & Security Test Template Pack (JSON-RPC)

5.31.1 Amaç

Bu paket; Kanboard JSON-RPC API için **negative test**, **authentication**, **authorization/RBAC** ve **brute-force koruması** kontrollerini standartlaştırır. Kanboard API, JSON-RPC 2.0 protokolünü kullanır.

5.31.2 Ön Koşullar

1. API endpoint erişilebilir olmalıdır: https://<SERVER>/jsonrpc.php.
2. Test için iki kimlik türü hazır olmalıdır:
    1. **Application API (jsonrpc + token)** → permissions kontrolü yapılmaz (RBAC testlerinde kullanılmaz).
    2. **User credentials (username + password veya personal access token)** → RBAC davranışı bu modda doğrulanır.
3. Eğer kullanıcıda 2FA etkinse API çağrıları için API key/personal token kullanılmalıdır (dokümantasyon notu).

5.31.3 JSON-RPC Protokol Negatifleri (Protocol-level)

Bu bölüm Kanboard'dan bağımsız, JSON-RPC 2.0 protokol validasyonudur.

5.31.3.1 Parse Error (-32700) – Invalid JSON

Request (bilerek bozuk JSON):

```
curl -u "jsonrpc:<JSONRPC_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"getAllProjects","id":1' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: Response içinde JSON-RPC error code **-32700 Parse error** ve id:null.

5.31.3.2 Invalid Request (-32600) – Request Object geçersiz

Request (jsonrpc alanı yok / method tipi yanlış vb.):

```
curl -u "jsonrpc:<JSONRPC_TOKEN>" \
  -d '{"method": 1, "params": "x"}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: -32600 Invalid Request ve id:null.

5.31.3.3 Method not found (-32601) – Geçersiz method

Request:

```
curl -u "jsonrpc:<JSONRPC_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"methodDoesNotExist","id":2}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: -32601 Method not found.

5.31.3.4 Invalid params (-32602) – Parametre yapısı yanlış

Request (beklenen params yerine farklı tip):

```
curl -u "jsonrpc:<JSONRPC_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"getAllTasks","id":3,"params":"wrong-type"}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: -32602 Invalid params (veya Kanboard tarafında "Result on failure: false" kalıbı ile result:false davranışı); gerçek çıktı "Expected Result" olarak kaydedilir.

Not: JSON-RPC'de params alanı **Array veya Object** olmalıdır; by-position veya by-name kullanılabilir.

5.31.4 Authentication Negatifleri (HTTP Basic / Header)

5.31.4.1 Yanlış Basic Auth – Unauthorized

Request:

```
curl -u "jsonrpc:WRONG_TOKEN" \
  -d '{"jsonrpc":"2.0","method":"getAllProjects","id":10}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: HTTP katmanında auth başarısız olmalı (Unauthorized). Doğrulama, kullanılan web server/PHP yapılandırmasına göre HTTP response olarak raporlanır. (Auth yönteminin HTTP Basic olduğu doğrulanır.)

5.31.4.2 User Credentials – Password veya Personal Access Token

Kural: User API çağrıları username + (password veya personal access token) ile yapılabilir.
Doğrulama: Aynı kullanıcıyla password ve token ile ayrı ayrı çağrı yapılıp sonuç tutarlılığı kontrol edilir.

5.31.4.3 Custom Header ile Authentication (Opsiyonel)

Kural: Basic Auth yerine header ile auth yapılandırılabilir (config ile).
Doğrulama: Header açık/kapalı durumlarında isteklerin beklenen şekilde kabul/red edilmesi.

5.31.5 Authorization / RBAC Negatifleri (Permission Denial)

Temel Kural: Application API (jsonrpc token) ile "permissions checked değildir"; RBAC testleri **User credentials** ile yapılmalıdır.

5.31.5.1 RBAC Denial – "Result on failure: false" doğrulaması

Kanboard API prosedürleri dokümantasyonda çoğunlukla "Result on failure: false" olarak tanımlıdır.

Test Şablonu (örnek: Project viewer ile task create denemesi):

1. Kullanıcı rolü: project-viewer (veya testte izinleri kısıtlı rol).
2. User API ile createTask çağrısı yapılır.
3. **Beklenen:**
    1. API response: result:false **ve** veride değişiklik olmaması (task oluşmamalı).
    2. UI üzerinden de aynı kullanıcının ilgili aksiyonu yapamaması (UI permission).

Request örneği:

```
curl -u "<VIEWER_USER>:<PASSWORD_OR_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"createTask","id":20,"params":{"title":"RBAC-
NEG","project_id":1}}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: result:false ve task oluşmaması.

5.31.5.2 "My…" Method Kısıtı (Application API ile)

Kural: Application API ile "My…" ile başlayan prosedürlere erişim yoktur (örn. getMe, getMyProjects).
Test: jsonrpc:<token> ile getMe çağrısı.
Beklenen: Method erişimi reddedilmeli (çıktı şekli ortamda gözlemlenerek kaydedilir: JSON-RPC error veya result:false).

5.31.6 Data Validation Negatifleri (Business/API-level)

Bu bölümdeki negatifler “protokol değil”, Kanboard prosedürlerinin “failure:false” davranışını doğrular.

5.31.6.1 Invalid ID – Non-existing entity

Örnek: closeTask invalid task_id

```
curl -u "jsonrpc:<JSONRPC_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"closeTask","id":30,"params":{"task_id":999999}}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: result:false.

5.31.6.2 Missing required param (by-name)

Örnek: getAllTasks param eksik

```
curl -u "jsonrpc:<JSONRPC_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"getAllTasks","id":31,"params":{"status_id":1}}' \
  https://<SERVER>/jsonrpc.php
```

Beklenen: JSON-RPC Invalid params (-32602) veya result:false (dokümantasyondaki “Result on failure: false” doğrulanır, gerçek çıktı kaydedilir).

5.31.6.3 Large attachment – createTaskFile sınırı (Environment-driven)

Kural: createTaskFile için max boyut PHP konfigürasyonuna bağlıdır; büyük dosyalar için uygun değildir.
Test: Boyutu PHP limitini aşan base64 payload ile upload denemesi.
Beklenen: result:false veya server error; gerçek çıktı evidence olarak raporlanır.

5.31.7 Brute Force Protection (Admin/Security)

Kural Seti (Login form):

1. Aynı kullanıcı adına **3 hatalı giriş** sonrası login form CAPTCHA gösterir.
2. **6 hatalı giriş** sonrası hesap **15 dk kilitlenir**.

User API özel notu:

1. User API üzerinden **3 başarısız auth denemesi** sonrası hesap, **login form üzerinden unlock** edilmeden kullanılamaz.

5.31.7.1 Test Şablonu – User API üzerinden 3 başarısız deneme

1. User API ile aynı kullanıcı için 3 kez yanlış password/token ile istek gönderilir.
2. Sonra doğru credential ile aynı kullanıcıyla istek tekrarlanır (örn. getMe).
3. **Beklenen:** Hesap unlock akışı gerektirecek şekilde davranmalı (dokümantasyona uyum).

4. Unlock işlemi **login form** üzerinden yapılır; sonrasında tekrar doğru credential ile API çağrısı doğrulanır.

Bu test yalnızca **test ortamında**, yetkili kapsamda uygulanır; üretim hesapları üzerinde yürütülmez.

5.31.8 Minimum Evidence (Kanıt Seti)

Her negatif/security test için aşağıdaki kanıtlar saklanır:

1. Request (curl/collection)
2. Response body (JSON veya HTML)
3. HTTP status + response headers (özellikle auth testlerinde)
4. DB/UI doğrulaması (veri değişmedi kanıtı)
5. Test user/role, timestamp, environment

5.32 Ek – Operasyonel Negatifler (Cronjob/CLI) – Kısa Paket

5.32.1 Daily Background Job zorunluluğu

Kanboard'un düzgün çalışması için günlük background job gerekir; cron ile ./cli cronjob çalıştırılması dokümante edilmiştir.

5.32.2 CLI'nin sadece local makinede çalışması

CLI komutları web server process'leri dışında, **local machine** üzerinde çalıştırılmak üzere tasarlanmıştır.

5.32.3 Test Şablonu

1. ./cli cronjob çalıştırılır, log/evidence alınır.
2. (Varsa) ./cli trigger:tasks ile "daily cronjob event" tetiklenir, açık task'lar üzerinde event davranışı doğrulanır.

5.33 Ek – RBAC Matrix (Role × Action)

Bu matris, Kanboard'ta **Uygulama Rolleri (Application roles)** ve **Proje Rolleri (Project roles)** ayrımına göre hazırlanmıştır. "Personal project"lerde **user management/permissions yoktur**; erişim yalnız proje sahibi ve administrator seviyesindedir.

5.33.1 Rol Tanımları (Referans)

Application roles

1. **Administrator:** her şeye erişim

2. **Manager:** team project oluşturabilir, application settings değiştiremez

3. **User:** yalnız personal project oluşturabilir

Project roles (team project içinde)

1. **Project Manager:** project settings değiştirebilir; Gantt ve reports erişimi

2. **Project Member:** task oluşturabilir ve board kullanabilir

3. **Project Viewer:** board ve task'lara read-only erişim

Custom project roles (özel roller)

1. Varsayılan olarak **Project Member** rolünden türetilir ve task create / close-open / move gibi kısıtlar uygulanabilir.

5.33.2 Legend

1. **A (Allowed):** izinli

2. **D (Denied):** yasak

3. **C (Conditional):** koşullu (ör. yalnız proje sahibi/manager/admin, veya custom role restriction'a bağlı)

4. **V (Validation Item):** dokümanda açıkça belirtilmedi; test/POC ile netleştirilecek

---

5.33.3 Application-level RBAC Matrix (Uygulama Seviyesi)

Action (Application Scope)	**Administrator**	**Manager**	**User**	**Notlar**
Create **Team Project**	A	A	D	Sadece Administrator/Manager.
Create **Personal Project**	A	A	A	"Anybody can create a personal project".
Change **Application settings**	A	D	D	Manager "cannot change application settings".
Add New User / Edit Any User / Remove User	A	D	D	"To add/remove a user, you must be an administrator."
Create Groups / Assign Users to Groups	A	D	D	"Only administrators can create new groups and assign users."
Unlock locked user (UI)	A	V	V	Admin unlock UI'da belirtilir;

Action (Application Scope)	Administrator	Manager	User	Notlar
				diğer roller env'e göre doğrulanır.
Brute force protection behavior (login form & user API)	A (admin can manage/unlock)	N/A	N/A	CAPTCHA/lock ve "user API 3 fail → login form unlock" kuralı.

5.33.4 Project-level RBAC Matrix (Team Project İçinde)

Aşağıdaki tablo **Project Role** üzerinden değerlendirilir (aynı kullanıcı farklı projelerde farklı project role alabilir). "Custom project roles" ile **Project Member** yetkileri kısıtlanabilir.

Action (Project Scope)	Project Manager	Project Member	Project Viewer	Notlar
View Board / Tasks	A	A	A	Viewer read-only.
Create Task	A	A	D	Member "create tasks". Custom role ile kısıtlanabilir.
Update Task (title/desc/fields)	A	C	D	Dokümanda member için "edit" açık yazmıyor; default davranış + custom role ile netleştirilecek.
Move Task (drag&drop / moveTaskPosition)	A	C	D	Custom role ile "Moving a task is not permitted" veya column-to-column kısıtları uygulanabilir.
Close/Open Task	A	C	D	Custom role ile "Closing or opening a task is not permitted" uygulanabilir.
Create/Update Subtask	A	C	D	Member default + restriction yoksa; net davranış V/C olarak doğrulanır.
Time Tracking (estimate/spent)	A	C	D	Default member davranışı doğrulanır (UI+API).
Start/Stop Subtask Timer	A	C	D	Timer feature için member default doğrulanır (UI+API).
Assign Users to Task (Assignee)	A	C	D	getAssignableUsers: "all members except viewers" → viewer assignable değil.

Action (Project Scope)	Project Manager	Project Member	Project Viewer	Notlar
Tags (assign tags to tasks)	A	C	D	Task-level tag atama; member default doğrulanır (UI+API).
Task Links (internal links create/remove)	A	C	D	Member default doğrulanır (UI+API).
Upload Attachments / Screenshots	A	C	D	Member default doğrulanır; viewer read-only.
Search / Custom Filters create	A	C	D	Custom filters proje bazlı; PM paylaşabilir.
Manage Project Settings (columns, categories, swimlanes, etc.)	A	D	D	"Project Manager can change project settings".
Manage Automatic Actions (create/remove actions)	A	D	D	Project settings kapsamında değerlendirilir.
Manage Custom Project Roles / Restrictions	A	D	D	Project settings > custom roles.
Project Permissions (add/remove users, assign roles)	A	D	D	Role assignment "Project Settings > Permissions"; group authorization PM tarafından yapılır.
Enable/Disable Public Access (public board/rss/ical)	A	D	D	Project settings kapsamında; token yenilenir.
View Gantt / Reports	A	V	V	PM erişimi dokümanda var; member/viewer erişimi ortamda doğrulanacak.
Remove Project	C	D	D	"To remove a project, you must be the manager of the project or an administrator."

5.33.5 RBAC İçin Zorunlu Validation Items (Test ile Netleştirilecek)

1. **Project Member** için "Update task / subtask / time tracking / attachments / tags / links" varsayılan izinleri (UI + User API)
2. **Gantt/Reports** erişimi: Member ve Viewer'ın varsayılan davranışı

3. “Personal project” üzerinde (owner dışı) erişim davranışı ve admin override doğrulaması

5.34 Ek – RBAC Test Case Pack (UI + User API)

Not: RBAC doğrulaması **User API** (username + password veya personal access token) ile yapılmalıdır; çünkü User API’de “application roles ve project permissions her prosedürde kontrol edilir”.
Application API (jsonrpc token) için “no permissions are checked” olduğundan RBAC için referans alınmaz.

5.34.1 Ortak Ön Koşullar (Tüm RBAC testleri için)

Test kullanıcı seti

1. U_ADMIN = Administrator
2. U_MANAGER = Manager
3. U_USER = User
4. U_PM = Project Manager (Team project içinde)
5. U_MEM = Project Member (Team project içinde)
6. U_VIEW = Project Viewer (Team project içinde)
7. U_CUST = Custom Project Role atanacak kullanıcı (role, “Project Member”dan türetilir)

Proje seti

1. TP1 = Team Project (çok kullanıcılı proje)
2. PP1 = Personal Project (tek kişilik proje)

Rol tanımları referansı

1. Project Manager: proje ayarlarını değiştirebilir, Gantt ve raporlara erişebilir
2. Project Member: görev oluşturabilir ve panoyu kullanabilir
3. Project Viewer: board/task read-only

API Auth

1. Endpoint: https://<SERVER>/jsonrpc.php
2. User credentials: username + (password veya personal access token)

5.34.2 Test Case Formatı (Standart)

Her test case şu kanıtları üretir:

1. UI ekran görüntüsü (role ile görünür/görünmez aksiyon)
2. API request/response (curl veya collection çıktısı)
3. Veri doğrulama (task oluştu/oluşmadı, status değişti/değişmedi vb.)

5.34.3 Application-level RBAC Testleri

TC-APP-01 – Team Project oluşturma yetkisi (Admin/Manager allowed, User denied)

Referans: Team project'i sadece Administrator ve Application Manager oluşturabilir; personal project'i herkes oluşturabilir.

Given

1. U_ADMIN, U_MANAGER, U_USER aktif.

When

1. UI: U_ADMIN ile "New project" üzerinden **Team Project** oluştur.
2. UI: U_MANAGER ile "New project" üzerinden **Team Project** oluştur.
3. UI: U_USER ile "New project" ekranında **Team Project** oluşturmayı dene.

Then

1. Admin: Team Project oluşturur (success).
2. Manager: Team Project oluşturur (success).
3. User: Team Project oluşturamaz (UI opsiyon yok / işlem engellenir).

TC-APP-02 – Application settings erişimi (Sadece Admin)

Referans: Manager "application settings değiştiremez".

Given

1. U_ADMIN ve U_MANAGER mevcut.

When

1. UI: U_ADMIN ile Application settings sayfasına gir.
2. UI: U_MANAGER ile Application settings sayfasına gir (menüden veya doğrudan URL).

Then

1. Admin: erişir (success).
2. Manager: erişemez (forbidden/redirect/menüde görünmeme).

TC-APP-03 – User/Group management (Sadece Admin create)

Referans: "To add a new user, you must be an administrator." + "Only administrators can create new groups and assign users."

Given

1. U_ADMIN, U_MANAGER mevcut.

When

1. UI: U_ADMIN ile User Management → New user oluştur.
2. UI: U_MANAGER ile User Management alanına gidip New user oluşturmayı dene.
3. UI: U_ADMIN ile Group oluştur ve kullanıcı ata.
4. UI: U_MANAGER ile Group oluşturmayı dene.

Then

1. Admin: user ve group create/assign yapar (success).
2. Manager: user create ve group create/assign yapamaz (opsiyon yok / engellenir).

5.34.4 Project-level RBAC Testleri (Team Project – TP1)

TC-PROJ-01 – Project Viewer read-only doğrulaması (UI + API: task create denied)

Referans: Project Viewer "board ve task'lara salt okunur erişim".
API referans: createTask success=task_id, failure=false.
User API permissions checked:

Given

1. TP1 var.
2. U_VIEW TP1'de **project-viewer** role ile atanmış.
3. API için U_VIEW credentials hazır.

When

1. UI: U_VIEW ile TP1 board aç → "New task / create" aksiyonunu dene.
2. API (User API): createTask ile task oluşturmaya çalış:

```
curl -u "<U_VIEW>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"createTask","id":3401,"params":{"title":"RBAC_VIEW_CREATE","project_id":<TP1_PROJECT_ID>}}' \
  https://<SERVER>/jsonrpc.php
```

Then

1. UI: task oluşturma mümkün olmaz (read-only).
2. API: result:false döner.
3. Veri: TP1'de "RBAC_VIEW_CREATE" task'ı oluşmamış olmalı.

TC-PROJ-02 – Project Member task create (UI + API: allowed)

Referans: Project Member "görevler oluşturabilir ve panoyu kullanabilir".
API referans: createTask success=task_id.

Given

1. U_MEM TP1'de **project-member**.
2. API için U_MEM credentials hazır.

When

1. UI: U_MEM ile TP1 → New task oluştur.
2. API (User API): createTask:

```
curl -u "<U_MEM>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"createTask","id":3402,"params":{"title":"RBAC_MEM_CREATE","project_id":<TP1_PROJECT_ID>}}' \
  https://<SERVER>/jsonrpc.php
```

Then

1. UI: task oluşur.
2. API: result numeric bir task_id döner.
3. Veri: task board'da görünür olmalı.

TC-PROJ-03 – Project Viewer assignable user list'te çıkmamalı (getAssignableUsers)

Referans: getAssignableUsers "all members except viewers".

Given

1. TP1'de U_MEM (member) ve U_VIEW (viewer) var.

When

1. API (User API, herhangi yetkili kullanıcıyla):

```
curl -u "<U_PM>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"getAssignableUsers","id":3403,"params":["<TP1_PROJECT_ID>"]}' \
  https://<SERVER>/jsonrpc.php
```

Then

1. Response dictionary içinde U_VIEW user_id **bulunmamalı**.
2. U_MEM user_id bulunmalı.

TC-PROJ-04 – Project Manager project settings erişimi (Member/Viewer denied)

Referans: Project Manager proje ayarlarını değiştirebilir.

Given

1. U_PM = project-manager
2. U_MEM = project-member
3. U_VIEW = project-viewer

When

1. UI: U_PM → Project settings (TP1) aç.
2. UI: U_MEM → Project settings açmayı dene.
3. UI: U_VIEW → Project settings açmayı dene.

Then

1. PM: erişir ve ayar menülerini görür.
2. Member/Viewer: erişemez veya yetkisi olmadığı şekilde kısıtlı görünür.

TC-PROJ-05 – Custom role: "Task creation is not permitted"

Referans: Custom project role, project member'dan türetilir; "Task creation is not permitted" restriction vardır.

Given

1. TP1'de U_CUST project-member seviyesinde projeye ekli.

2. U_PM (veya yetkili kişi) Project settings → Custom Roles ekranına erişebiliyor.

When

1. UI (PM): Custom Roles → yeni rol oluştur: CR_NO_CREATE.
2. UI (PM): bu role **Project restriction** olarak "Task creation is not permitted" ekle.
3. UI (PM): Project settings → Permissions → U_CUST için role CR_NO_CREATE ata.
4. UI: U_CUST ile New task oluşturmayı dene.
5. API (User API): U_CUST ile createTask dene (TC-PROJ-02 örneği).

Then

1. UI: task oluşturma engellenir.
2. API: createTask → result:false.
3. Veri: yeni task oluşmaz.

TC-PROJ-06 – Custom role: "Closing or opening a task is not permitted"

Referans: Restriction listesinde "Closing or opening a task is not permitted" bulunur.
API referans: closeTask/openTask success=true, failure=false.

Given

1. TP1'de en az 1 açık task: <TASK_ID>.
2. U_CUST için custom role: CR_NO_STATUS_CHANGE hazırlanacak.

When

1. UI (PM): Custom Roles → CR_NO_STATUS_CHANGE oluştur ve "Closing or opening a task is not permitted" restriction ekle.
2. UI (PM): Permissions → U_CUST'a CR_NO_STATUS_CHANGE ata.
3. UI: U_CUST ile task'ı close/open yapmayı dene.
4. API (User API): U_CUST ile closeTask ve openTask dene:

```
curl -u "<U_CUST>:<PASSWORD_OR_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"closeTask","id":3406,"params":{"task_id":<TASK_ID>}}' \
  https://<SERVER>/jsonrpc.php
```

```
curl -u "<U_CUST>:<PASSWORD_OR_TOKEN>" \
  -d '{"jsonrpc":"2.0","method":"openTask","id":3407,"params":{"task_id":<TASK_ID>}}' \
  https://<SERVER>/jsonrpc.php
```

Then

1. UI: close/open işlemi engellenir.
2. API: closeTask/openTask → result:false.
3. Veri: task status'ü değişmemiş olmalı.

TC-PROJ-07 – Custom role: "Moving a task is not permitted"

Referans: Restriction listesinde "Moving a task is not permitted" vardır.
API referans: moveTaskPosition success=true, failure=false.

Given

1. TP1'de move edilebilir bir task: <TASK_ID>.
2. U_CUST için custom role: CR_NO_MOVE.

When

1. UI (PM): Custom Roles → CR_NO_MOVE oluştur ve "Moving a task is not permitted" ekle.
2. UI (PM): Permissions → U_CUST'a CR_NO_MOVE ata.
3. UI: U_CUST ile drag&drop ile task'ı başka kolona taşımayı dene.
4. API (User API): U_CUST ile moveTaskPosition dene:

```
curl -u "<U_CUST>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"moveTaskPosition","id":3408,"params":{"project_id":<TP1_PROJECT
_ID>,"task_id":<TASK_ID>,"column_id":<TARGET_COLUMN_ID>,"position":1,"swimlane_id":<SWI
MLANE_ID>}}' \
  https://<SERVER>/jsonrpc.php
```

Then

1. UI: move engellenir.
2. API: result:false.
3. Veri: task column/position değişmemiş olmalı.

TC-PROJ-08 – Project Permissions API: addProjectUser / changeProjectUserRole / removeProjectUser

Referans: addProjectUser, changeProjectUserRole, removeProjectUser prosedürleri ve success/failure sonuçları.

Given

1. TP1 var.
2. U_PM veya U_ADMIN ile yetkili API çağrısı yapılabiliyor.
3. Eklenecek kullanıcı: U_USER2 (project'e dahil değil).

When

1. API: addProjectUser ile U_USER2'yi project-viewer ekle.
2. API: changeProjectUserRole ile role'ü project-member yap.
3. API: removeProjectUser ile project erişimini kaldır.

Örnek çağrılar

```
curl -u "<U_PM>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"addProjectUser","id":3409,"params":["<TP1_PROJECT_ID>","<U_USE
R2_ID>","project-viewer"]}' \
  https://<SERVER>/jsonrpc.php
```

```
curl -u "<U_PM>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"changeProjectUserRole","id":3410,"params":["<TP1_PROJECT_ID>","
<U_USER2_ID>","project-member"]}' \
  https://<SERVER>/jsonrpc.php
```

```
curl -u "<U_PM>:<PASSWORD_OR_TOKEN>" \
  -d
'{"jsonrpc":"2.0","method":"removeProjectUser","id":3411,"params":[<TP1_PROJECT_ID>,<U_US
ER2_ID>]}' \
  https://<SERVER>/jsonrpc.php
```

Then

1. Her çağrı success: result:true.
2. UI doğrulama:
    1. U_USER2 viewer iken read-only görmeli,
    2. member iken task create yapabilmeli,
    3. remove sonrası projeyi görmemeli.

TC-PROJ-09 – Proje silme yetkisi (Project manager veya Admin)

Referans: "To remove a project, you must be the manager of the project or an administrator."

Given

1. Silinecek test projesi: TP_DEL.

When

1. UI: U_PM ile TP_DEL → Project settings → Remove.
2. UI: U_MEM ile aynı aksiyonu dene.
3. UI: U_ADMIN ile remove dene.

Then

1. PM/Admin: remove yapabilir.
2. Member: remove yapamaz.
3. Remove sonrası: proje ve bağlı task'lar sistemde görünmemeli.

5.34.5 Personal Project RBAC Testleri (PP1)

TC-PERS-01 – Personal project erişimi (Owner + Admin)

Referans: Personal project'te user management yoktur; sadece owner ve administrators erişebilir; personal projects permissions tanımlayamaz.

Given

1. PP1 sahibi: U_USER.
2. Başka bir normal kullanıcı: U_USER2.
3. Admin: U_ADMIN.

When

1. UI: U_USER ile PP1 görünür mü ve açılabilir mi kontrol et.
2. UI: U_USER2 ile PP1 dashboard/listing'de görünüyor mu kontrol et; direct URL ile açmayı dene.
3. UI: U_ADMIN ile PP1'e erişmeyi dene.

Then

1. Owner: erişir.
2. Admin: erişir.
3. Diğer kullanıcı: erişemez / projeyi görmez.

5.34.6 API Modu Doğrulama (RBAC açısından kritik)

TC-API-01 – Application API "My…" prosedürlerine erişemez

Referans: Application API “No access to procedures that start with ‘My…’ (getMe/getMyProjects)”.

Given

1. Application API credentials: jsonrpc:<TOKEN>.

When

1. API: getAllProjects çağır (expect success).
2. API: getMe veya getMyProjects çağır (expect denied).

Then

1. getAllProjects: success.
2. getMe/getMyProjects: başarısız olmalı (çıktı türü environment’da kaydedilir: JSON-RPC error veya result:false).

5.34.7 Validation Items Listesi (Test ile kesinleştirilecek)

Aşağıdaki aksiyonların “member allowed” davranışı dokümanda satır satır enumerasyon olarak geçmediği için (özellikle update/edit kapsamı), **UI + User API** ile kesinleştirilecek:

1. Project Member için updateTask, removeTask, attachments, tags, internal links, subtask update ve time tracking default izinleri (Allowed/Denied sonucu matrise işlenecek).
2. Gantt/Reports erişimi: member/viewer için (UI doğrulama + matrise net karar).